“Title of the Project”

Placement Hub

Data Structures and Algorithms -3 (25CS2103E)

in

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING

by

R C SAI SHARAN : 2520030128

KOMPELLI SAANVI : 2520030083

S KETANA SINDHU : 2520030487

Team Number : 21

Section : 6

Under the Guidance of

Dr. V. Sireesha

Associate Professor

Department of Computer Science and Engineering

2026-27

Koneru Lakshmaiah Education Foundation

(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)
Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.
Phone No: 7815926816, www.klh.edu.in

DECLARATION

We hereby declare that the project report entitled “Placement Hub” is a record of bona fide work carried out by R.C. Sai Sharan – 2520030128, Kompelli Saanvi – 2520030083, and S. Ketana Sindhu – 2520030487 in partial fulfillment of the requirements for the award of the degree of Bachelor of Technology in Computer Science and Engineering during the academic year 2026-27.

The work presented in this report is based on the design, implementation, testing, and evaluation of Placement Hub as a Data Structures and Algorithms-3 project. The implementation and observations presented are prepared by the project team for academic submission.

R C Sai Sharan – 2520030128

Kompelli Saanvi – 2520030083

S Ketana Sindhu – 2520030487

CERTIFICATE

This is to certify that the project report entitled “Placement Hub” is a bona fide work done and submitted by R.C. Sai Sharan – 2520030128, Kompelli Saanvi – 2520030083, and S. Ketana Sindhu – 2520030487 in partial fulfillment of the requirements for the award of the degree of Bachelor of Technology in the Department of Computer Science and Engineering during the academic year 2026-27.

The project has been carried out under the guidance of Dr. V. Sireesha, Associate Professor, Department of Computer Science and Engineering.

Signature of the Guide **Signature of the HOD**

(Dr. V. Sireesha) (Dr. P. Venkateshwara Rao)

Signature of the External Examiner

ACKNOWLEDGEMENT

We express our sincere gratitude to our guide, Dr. V. Sireesha, Associate Professor, Department of Computer Science and Engineering, for her valuable guidance, encouragement, and continuous support throughout the development of Placement Hub. Her suggestions helped us understand how Data Structures and Algorithms can be applied to a practical placement management problem.

We thank Dr. P. Venkateshwara Rao, Associate Professor and Head of the Department of Computer Science and Engineering, for providing the academic environment and facilities required for the completion of this project. We also thank the faculty members and classmates for their feedback and support during development and testing.

Finally, we thank everyone who contributed directly or indirectly to the successful completion of this project. The development of Placement Hub strengthened our skills in Java, web development, algorithm design, debugging, testing, documentation, and teamwork.

ABSTRACT

Placement Hub is a Java-based placement management system developed as a Data Structures and Algorithms-3 project. It provides students with a centralized platform to register, log in, view companies and job opportunities, search for jobs, check eligibility, receive recommendations, apply for suitable positions, and track applications.

The system demonstrates practical use of ArrayList, HashMap, Stack, Queue, Trie, and graph-based structures. Algorithms including KMP pattern matching, Rabin-Karp, edit distance, Damerau-Levenshtein distance, fuzzy search, sorting, job matching, recommendation, bipartite matching, and maximum-flow concepts are connected to placement operations.

The backend uses Java 21+ and the built-in HTTP server, while HTML, CSS, and JavaScript provide the frontend. Runtime data is maintained in Java collections rather than MySQL or JDBC. The demonstration dataset contains 25 companies and 50 jobs.

TABLE OF CONTENTS

	Chapter	Contents	Page No.
1	**Introduction**	1.1 Overview 1.2 Problem statement 1.3 Scope of the project 1.4 Objectives	1-11
2	**System Requirements**	2.1 Functional Requirements 2.2 Non-Functional Requirements 2.3 Hardware and Software Requirements 2.4 Existing System and Gap	12-17
3	**System Design**	3.1 High-level Architecture Diagram 3.2 Description of each layer 3.3 Data Structures and Algorithms 3.4 Data Flow and Matching	18-29
4	**Implementation**	4.1 Module-wise implementation 4.2 Front-end logic (UI rendering, state management) 4.3 API integration 4.4 DSA implementation and pseudocode	30-48
5	**Results**	5.1 Functional Testing 5.2 Search and Fuzzy Search Results 5.3 Recommendation and Eligibility Results 5.4 Application Workflow Results	49-66
6	**Challenges & Limitations**	6.1 Issues faced during development 6.2 Solutions applied 6.3 Limitations of the System	67-81
7	**Conclusion & Future Work**	7.1 Conclusion 7.2 Future Work 7.3 Skills learned during development	82-90
	References	Books, tutorials, APIs, documentation sites used	91-92

LIST OF FIGURES

Fig. 1.1 Problem-to-Solution Flow (p. 1)

Fig. 1.2 DSA Algorithm Pipeline (p. 2)

Fig. 1.3 Student-to-Job Matching Workflow (p. 3)

Fig. 1.4 Problem-to-Solution Flow (p. 4)

Fig. 1.5 Problem-to-Solution Flow (p. 5)

Fig. 1.6 Project Scope (p. 6)

Fig. 1.7 High-Level System Architecture (p. 7)

Fig. 1.8 Project Objectives (p. 8)

Fig. 1.9 Project Objectives (p. 9)

Fig. 1.10 Student-to-Job Matching Workflow (p. 10)

Fig. 1.11 Problem-to-Solution Flow (p. 11)

Fig. 2.1 System Requirements (p. 12)

Fig. 2.2 System Requirements (p. 13)

Fig. 2.3 System Requirements (p. 14)

Fig. 2.4 System Requirements (p. 15)

Fig. 2.5 System Requirements (p. 16)

Fig. 2.6 System Requirements (p. 17)

Fig. 3.1 High-Level System Architecture (p. 18)

Fig. 3.2 High-Level System Architecture (p. 18)

Fig. 3.3 High-Level System Architecture (p. 19)

Fig. 3.4 API Integration (p. 20)

Fig. 3.5 API Integration (p. 21)

Fig. 3.6 Fuzzy Search Workflow (p. 22)

Fig. 3.7 Frontend Interaction (p. 23)

Fig. 3.8 Data Structures Used (p. 24)

Fig. 3.9 System Requirements (p. 25)

Fig. 3.10 Fuzzy Search Workflow (p. 26)

Fig. 3.11 Recommendation Workflow (p. 27)

Fig. 3.12 Job Search Data Flow (p. 27)

Fig. 3.13 Student-to-Job Matching Workflow (p. 28)

Fig. 3.14 Student-to-Job Matching Workflow (p. 28)

Fig. 3.15 Job Search Data Flow (p. 29)

Fig. 4.1 Student-to-Job Matching Workflow (p. 30)

Fig. 4.2 Placement Hub Overview (p. 31)

Fig. 4.3 Placement Hub Overview (p. 32)

Fig. 4.16 Application Workflow (p. 45)

Fig. 4.17 Student-to-Job Matching Workflow (p. 45)

Fig. 4.18 Algorithmic Workflow (p. 46)

Fig. 4.19 Student-to-Job Matching Workflow (p. 47)

Fig. 4.20 Student-to-Job Matching Workflow (p. 48)

Fig. 5.1 System Requirements (p. 49)

Fig. 5.2 Testing Workflow (p. 50)

Fig. 5.3 Student-to-Job Matching Workflow (p. 51)

Fig. 5.4 Fuzzy Search Workflow (p. 52)

Fig. 5.5 Fuzzy Search Workflow (p. 53)

Fig. 5.6 Testing Workflow (p. 54)

Fig. 5.7 Student-to-Job Matching Workflow (p. 55)

Fig. 5.8 Recommendation Workflow (p. 56)

Fig. 5.9 Application Workflow (p. 57)

Fig. 5.10 Application Workflow (p. 58)

Fig. 5.11 Application Workflow (p. 59)

Fig. 5.12 System Requirements (p. 60)

Fig. 5.13 Fuzzy Search Workflow (p. 61)

Fig. 5.14 DSA Algorithm Pipeline (p. 62)

Fig. 5.15 Testing Workflow (p. 63)

Fig. 5.16 Student-to-Job Matching Workflow (p. 64)

Fig. 5.17 Problem-to-Solution Flow (p. 65)

Fig. 5.18 Student-to-Job Matching Workflow (p. 66)

Fig. 6.1 DSA Algorithm Pipeline (p. 67)

Fig. 6.2 Application Workflow (p. 68)

Fig. 6.3 Fuzzy Search Workflow (p. 69)

Fig. 6.4 Frontend Interaction (p. 70)

Fig. 6.5 System Requirements (p. 71)

Fig. 6.6 System Requirements (p. 72)

Fig. 6.7 DSA Algorithm Pipeline (p. 73)

Fig. 6.8 API Integration (p. 74)

Fig. 6.9 Application Workflow (p. 75)

Fig. 6.10 Development Challenges (p. 76)

Fig. 6.11 Development Challenges (p. 77)

Fig. 6.12 Development Challenges (p. 78)

Fig. 4.4 Placement Hub Overview (p. 33)

Fig. 4.5 Application Workflow (p. 34)

Fig. 4.6 Student-to-Job Matching Workflow (p. 35)

Fig. 4.7 Student-to-Job Matching Workflow (p. 36)

Fig. 4.8 Student-to-Job Matching Workflow (p. 37)

Fig. 4.9 API Integration (p. 38)

Fig. 4.10 Frontend Interaction (p. 39)

Fig. 4.11 Student-to-Job Matching Workflow (p. 40)

Fig. 4.12 DSA Algorithm Pipeline (p. 41)

Fig. 4.13 DSA Algorithm Pipeline (p. 42)

Fig. 4.14 Fuzzy Search Workflow (p. 43)

Fig. 4.15 System Requirements (p. 44)

Fig. 6.13 Development Challenges (p. 79)

Fig. 6.14 Project Objectives (p. 80)

Fig. 6.15 System Requirements (p. 81)

Fig. 7.1 Data Structures Used (p. 82)

Fig. 7.2 DSA Algorithm Pipeline (p. 83)

Fig. 7.3 High-Level System Architecture (p. 84)

Fig. 7.4 Application Workflow (p. 85)

Fig. 7.5 Recommendation Workflow (p. 86)

Fig. 7.6 Future Enhancement Roadmap (p. 87)

Fig. 7.7 Student-to-Job Matching Workflow (p. 88)

Fig. 7.8 DSA Algorithm Pipeline (p. 89)

Fig. 7.9 Project Objectives (p. 90)

1. INTRODUCTION — 1.1 Overview

Placement Hub is a web-based placement management system designed to organize campus placement activities for students and placement administrators. It brings student profiles, companies, job opportunities, search, eligibility, recommendation, and applications into one workflow.

The project is developed as a Data Structures and Algorithms-3 application. Its purpose is not only to provide placement features but also to demonstrate how algorithms can solve practical problems involving search, similarity, matching, ranking, and validation.

The system is organized around a student-centric placement workflow. A student's profile is used as the basis for eligibility checks and recommendations, while job records contain the academic and technical conditions required by each opportunity. This separation makes the application easier to maintain and also allows the DSA components to operate on clearly defined inputs.

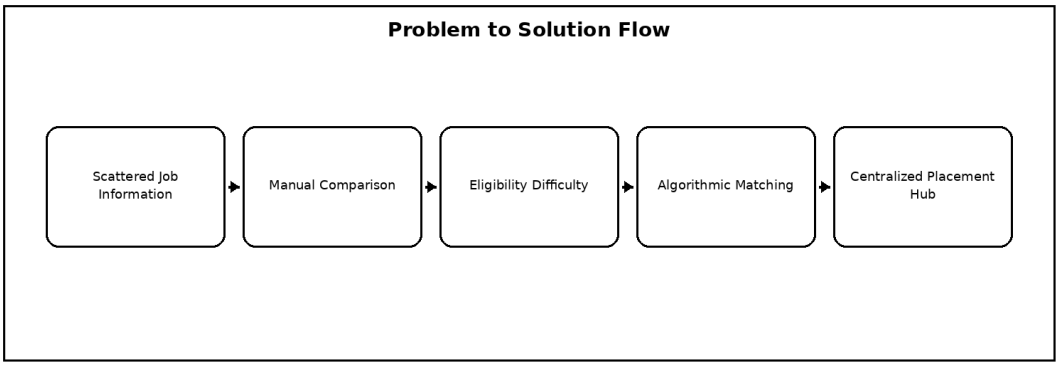


Fig. 1.1 Problem-to-Solution Flow

1. INTRODUCTION — 1.1 Overview

A student profile includes academic information, branch, backlog count, graduation year, and technical skills. A job profile contains role, company, package, minimum CGPA, allowed branches, required skills, maximum backlogs, drive date, and job description.

These records are connected through identifiers. The application uses DAO classes to manage them in Java collections so that the algorithmic operations remain independent from persistent database technology.

The system is organized around a student-centric placement workflow. A student's profile is used as the basis for eligibility checks and recommendations, while job records contain the academic and technical conditions required by each opportunity. This separation makes the application easier to maintain and also allows the DSA components to operate on clearly defined inputs.

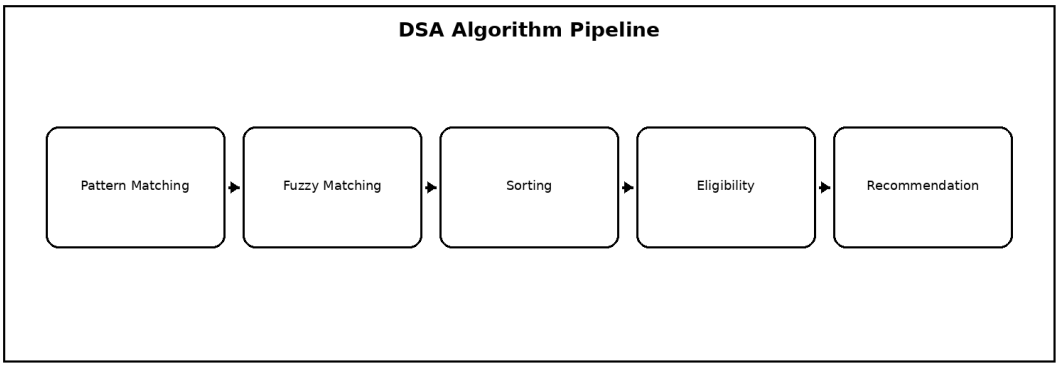


Fig. 1.2 DSA Algorithm Pipeline

1. INTRODUCTION — 1.1 Project Context

Traditional placement workflows can involve spreadsheets, notices, messages, and multiple portals. Students may need to manually inspect a large number of opportunities and compare eligibility conditions.

Placement Hub provides a centralized workflow in which job discovery, eligibility checking, recommendation, and application tracking are connected. This reduces repeated manual effort and gives DSA concepts a visible role in the application.

Recommendations are generated from the relationship between a student's profile and the available jobs. Academic constraints are considered before relevance is ranked. This approach avoids recommending opportunities that are clearly outside the student's eligibility and focuses the final list on actionable choices.

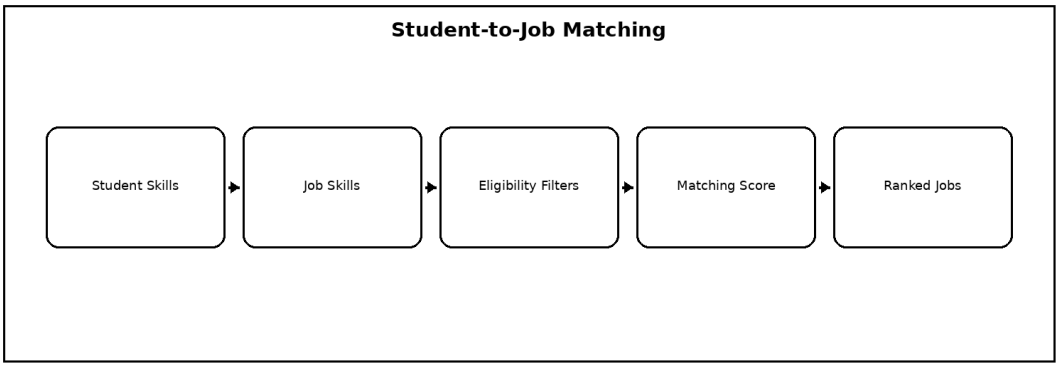


Fig. 1.3 Student-to-Job Matching Workflow

1. INTRODUCTION — 1.2 Problem Statement

The problem addressed by this project is to develop a placement management system that can organize placement information and efficiently match students with suitable job opportunities.

The system must consider academic constraints, branch restrictions, backlog limits, skills, and search relevance. It must also prevent invalid and duplicate application operations.

The project therefore treats placement discovery as a data organization and algorithmic decision problem. Instead of presenting a large undifferentiated list, the system filters and ranks opportunities using attributes such as branch, CGPA, backlog limits, skills, and search relevance. This makes the DSA concepts visible in a realistic application scenario.

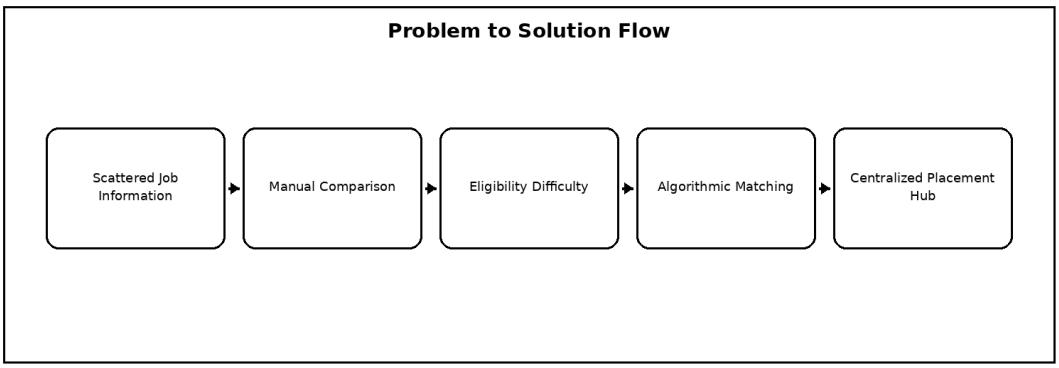


Fig. 1.4 Problem-to-Solution Flow

1. INTRODUCTION — 1.2 Problem Statement

Exact keyword search is not sufficient when a student makes a spelling mistake. Similarly, listing every job without ranking does not help a student identify the most suitable opportunities.

Therefore, Placement Hub incorporates fuzzy matching, pattern matching, sorting, eligibility checking, and recommendation logic. These operations form the algorithmic core of the system.

The project therefore treats placement discovery as a data organization and algorithmic decision problem. Instead of presenting a large undifferentiated list, the system filters and ranks opportunities using attributes such as branch, CGPA, backlog limits, skills, and search relevance. This makes the DSA concepts visible in a realistic application scenario.

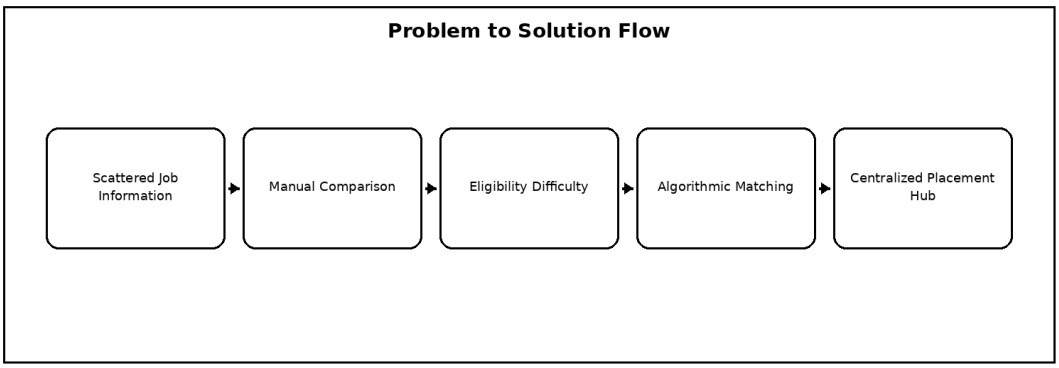


Fig. 1.5 Problem-to-Solution Flow

1. INTRODUCTION — 1.3 Scope of the Project

The project includes student registration and login, profile management, company listing, job listing, normal search, fuzzy search, sorting, eligibility checking, recommendation, job application, application viewing, and administrative operations.

The current scope is an academic prototype. It does not include live recruitment feeds, payment systems, enterprise authentication, or persistent cloud storage.

The scope covers the complete student-side workflow from registration to application tracking and also includes administrative views. The current implementation is intentionally focused on academic demonstration, so the data is maintained in memory during a server session. This design keeps the DSA operations transparent and avoids introducing an external persistence layer into the project.

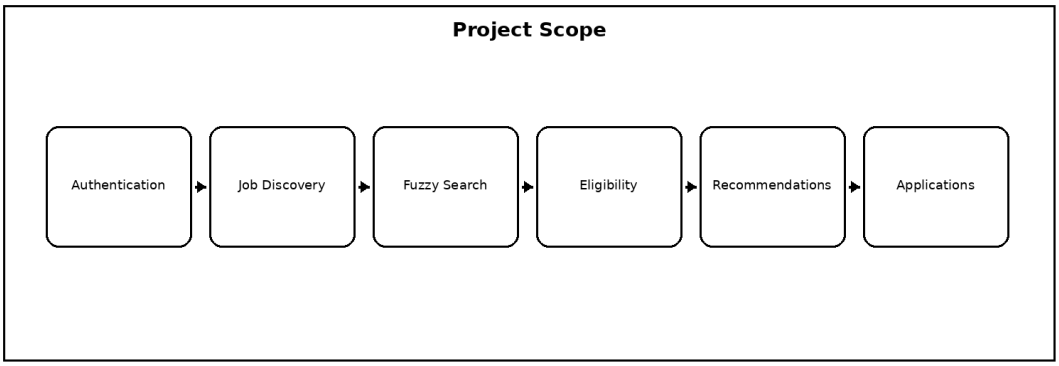


Fig. 1.6 Project Scope

1. INTRODUCTION — 1.3 Scope of the Project

The demonstration dataset contains 25 companies and 50 job opportunities. New runtime records can be added through DAO methods without changing the frontend workflow.

The architecture also leaves a clear path for future extensions such as database persistence, role-based access, analytics, and cloud deployment.

The scope covers the complete student-side workflow from registration to application tracking and also includes administrative views. The current implementation is intentionally focused on academic demonstration, so the data is maintained in memory during a server session. This design keeps the DSA operations transparent and avoids introducing an external persistence layer into the project.

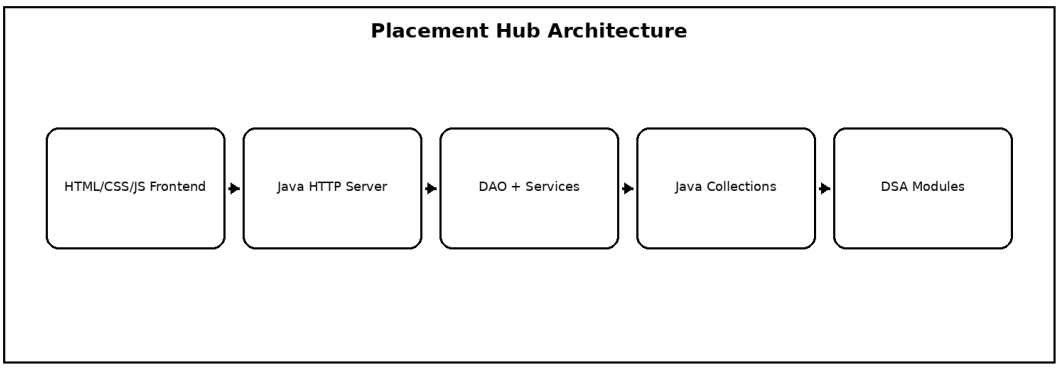


Fig. 1.7 High-Level System Architecture

1. INTRODUCTION — 1.4 Objectives

The first objective is to provide a simple placement interface through which students can discover and apply for suitable jobs. The second is to apply DSA techniques directly to useful placement operations.

A further objective is to create a modular Java application separating models, data access, services, APIs, and DSA algorithms.

A further objective is modularity. Each major responsibility is isolated so that an algorithm can be tested independently and then connected to the web workflow. This approach also makes it easier to extend the project with additional jobs, companies, matching rules, or algorithms without redesigning the complete application.

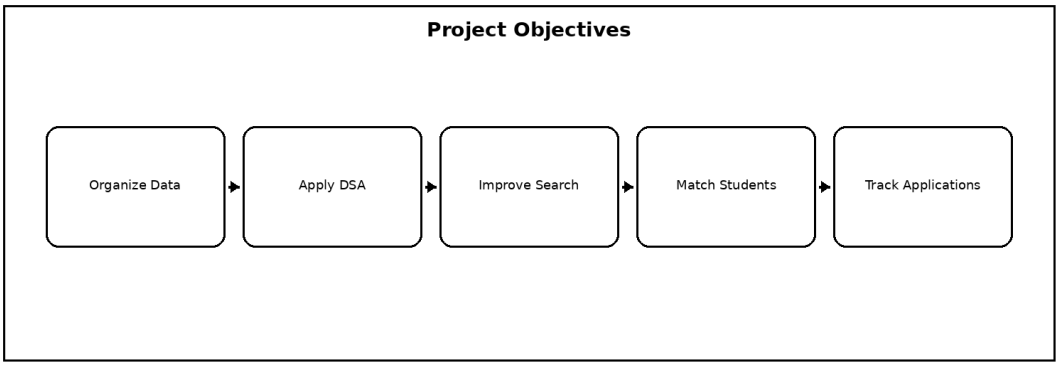


Fig. 1.8 Project Objectives

1. INTRODUCTION — 1.4 Objectives

The project also aims to provide typo-tolerant search through similarity measures. A user should receive useful results even when the query is not perfectly spelled.

Another objective is to demonstrate a complete workflow from student authentication through recommendation and application tracking without relying on an external database.

A further objective is modularity. Each major responsibility is isolated so that an algorithm can be tested independently and then connected to the web workflow. This approach also makes it easier to extend the project with additional jobs, companies, matching rules, or algorithms without redesigning the complete application.

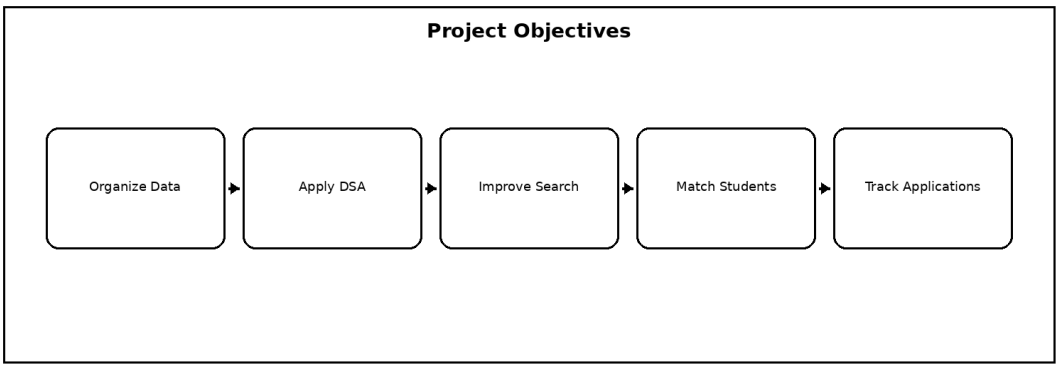


Fig. 1.9 Project Objectives

1. INTRODUCTION — Project Workflow

The student begins by registering and logging in. After authentication, the student can view companies and jobs, search for roles, check eligibility, inspect recommendations, and submit applications.

Each frontend action communicates with a Java API endpoint. The server invokes DAO, service, and DSA components and returns the processed result to the browser.

The frontend is designed around simple student actions. Forms collect input, dashboard sections expose major functions, and JavaScript calls the backend asynchronously. Results are rendered dynamically so that the user can search, inspect jobs, receive recommendations, and track applications without navigating through unrelated screens.

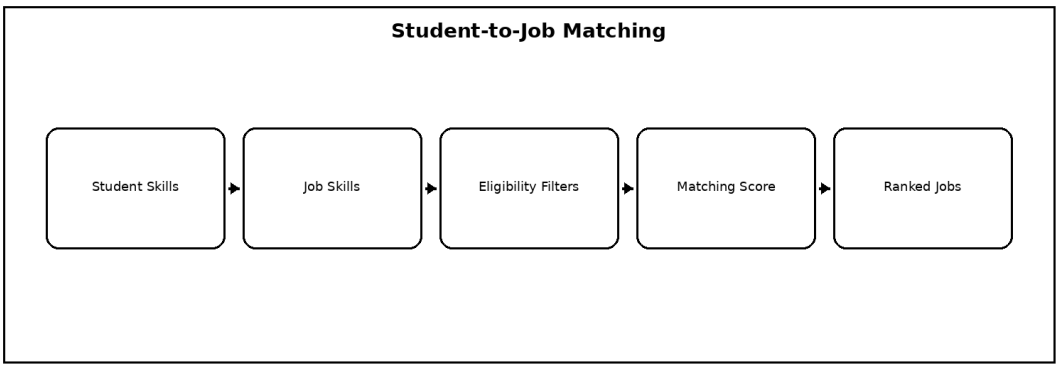


Fig. 1.10 Student-to-Job Matching Workflow

1. INTRODUCTION — Significance

Placement Hub shows how textbook algorithms can be connected to a realistic software problem. Pattern matching becomes job searching, string similarity becomes fuzzy search, and matching becomes recommendation and eligibility analysis.

The project therefore provides experience in both algorithmic problem solving and full application development.

The software stack was selected to keep the project easy to build and demonstrate. Java provides the object-oriented backend and DSA implementation, Maven handles compilation, and standard web technologies provide the user interface. The built-in Java HTTP server removes the need for an additional application server.

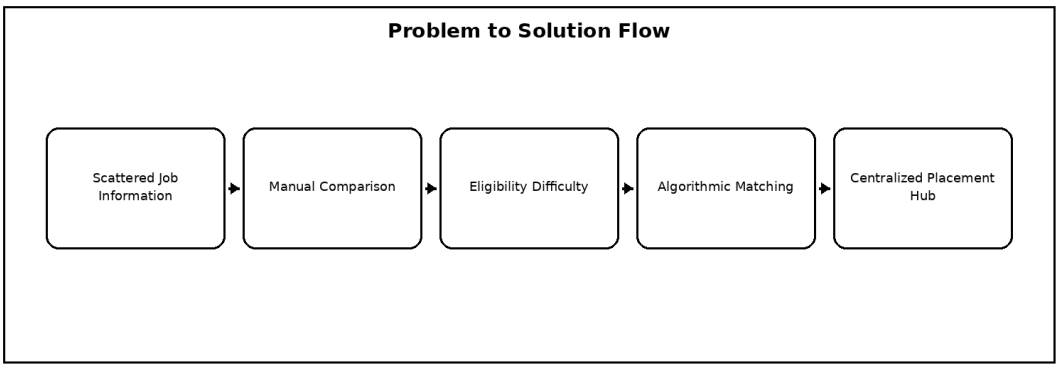


Fig. 1.11 Problem-to-Solution Flow

2. SYSTEM REQUIREMENTS — 2.1 Functional Requirements

The functional requirements describe the operations that the system must perform. Student registration creates a profile, while login authenticates an existing profile.

Company and job modules provide placement information. Job search, fuzzy search, sorting, eligibility, recommendation, and application operations form the main student workflow.

Functional requirements describe what the system must do from the user's perspective. They form the basis for API endpoints and frontend actions. Each function is connected to a backend operation so that the final application can be tested from input through processing to visible output.

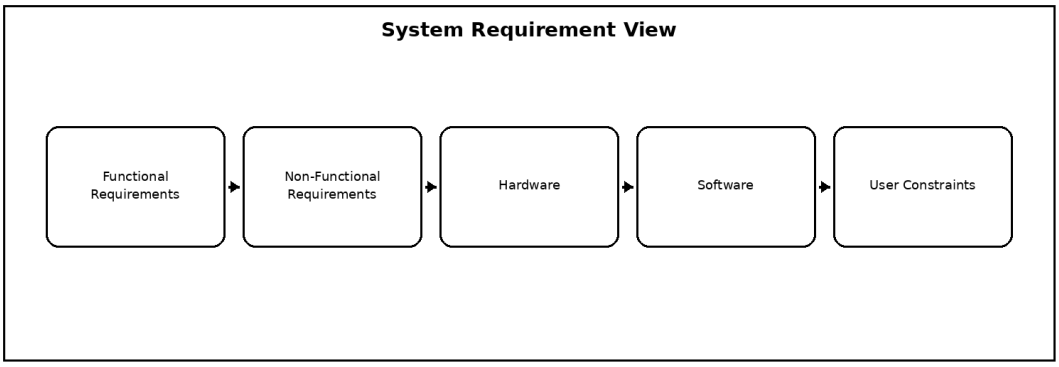


Fig. 2.1 System Requirements

2. SYSTEM REQUIREMENTS — 2.1 Functional Requirements

Eligibility checking compares CGPA, branch, backlogs, and skills against job constraints. Recommendation ranks suitable opportunities using compatibility information.

Application management creates application records and prevents duplicate submissions. Administrative functions provide views of project records and support status updates.

Functional requirements describe what the system must do from the user's perspective. They form the basis for API endpoints and frontend actions. Each function is connected to a backend operation so that the final application can be tested from input through processing to visible output.

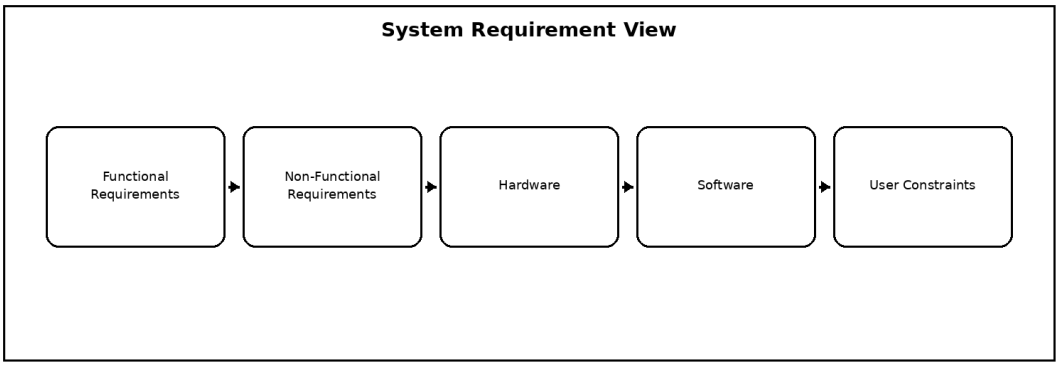


Fig. 2.2 System Requirements

2. SYSTEM REQUIREMENTS — 2.2 Non-Functional Requirements

Usability requires clear navigation, readable job information, understandable forms, and simple search interactions. Performance requires responsive operations for the demonstration dataset.

Reliability requires validation of missing records and invalid inputs. Maintainability is supported through separation into model, DAO, service, API, and DSA packages.

Functional requirements describe what the system must do from the user's perspective. They form the basis for API endpoints and frontend actions. Each function is connected to a backend operation so that the final application can be tested from input through processing to visible output.

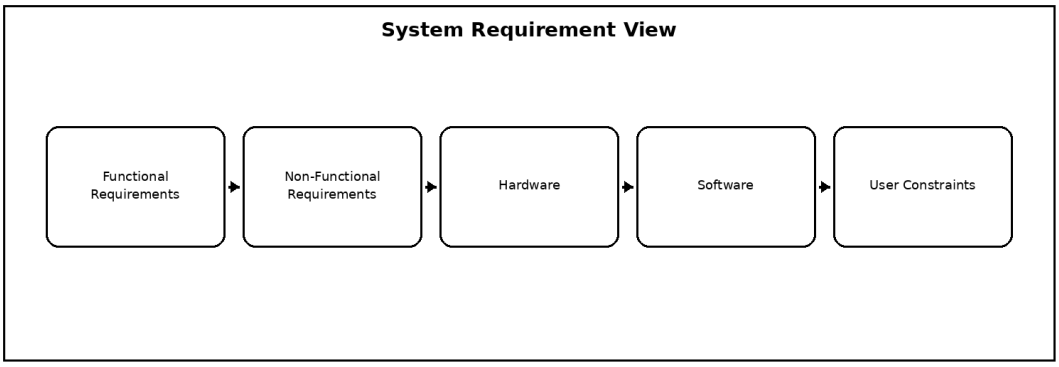


Fig. 2.3 System Requirements

2. SYSTEM REQUIREMENTS — 2.2 Non-Functional Requirements

Scalability in the academic prototype means that additional students, companies, jobs, and applications can be stored in collections during execution.

Portability is achieved because the system requires Java, Maven, and a browser rather than a specialized execution environment.

Functional requirements describe what the system must do from the user's perspective. They form the basis for API endpoints and frontend actions. Each function is connected to a backend operation so that the final application can be tested from input through processing to visible output.

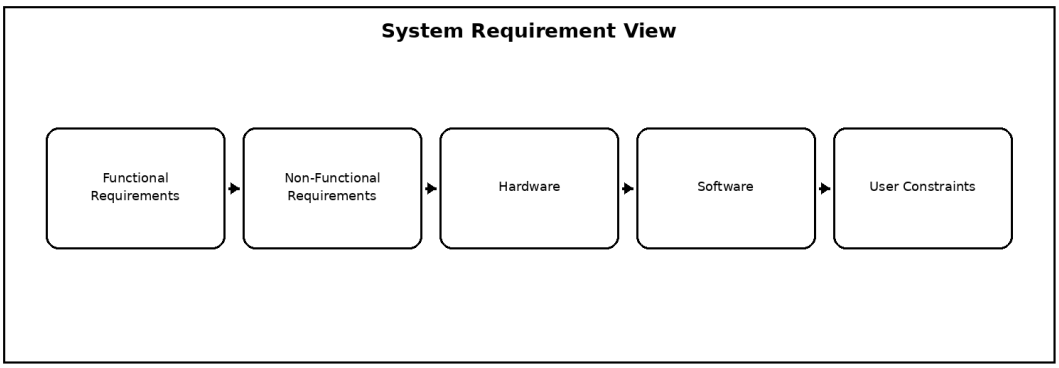


Fig. 2.4 System Requirements

2. SYSTEM REQUIREMENTS — 2.3 Hardware and Software Requirements

Recommended hardware includes a modern dual-core processor, at least 4 GB RAM, adequate storage, and a standard display. No specialized hardware is required.

Software includes JDK 21 or later, Apache Maven, an IDE such as Visual Studio Code or IntelliJ IDEA, and a modern browser.

The hardware requirements are modest because the application is executed locally and the project dataset is small enough to be handled comfortably by standard student development systems. No dedicated server or special accelerator is required for the current implementation.

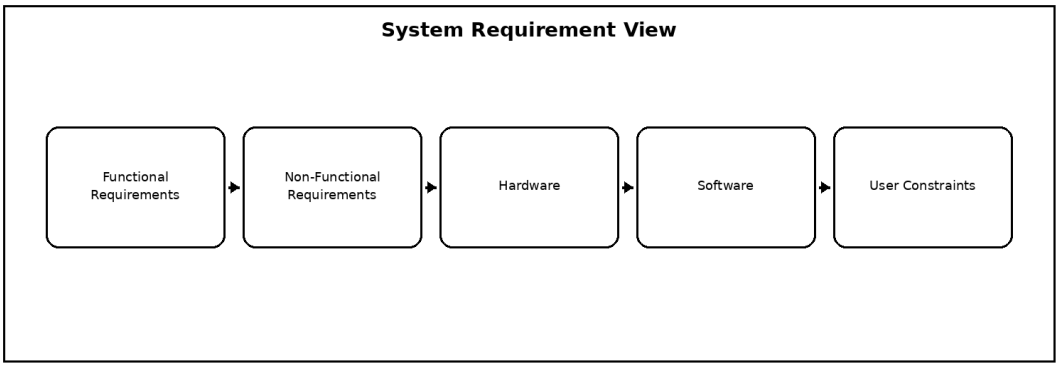


Fig. 2.5 System Requirements

2. SYSTEM REQUIREMENTS — 2.4 Existing System and Gap

Traditional placement processes may depend on spreadsheets and manual checking. As the number of opportunities grows, manual search and eligibility verification become inconvenient.

Placement Hub addresses this gap by combining placement records with search, fuzzy matching, recommendation, eligibility, and application tracking in one application.

The matching process can be viewed as a sequence of filters followed by ranking. Hard constraints such as branch, CGPA, and backlog limits remove unsuitable opportunities first. Skill similarity then contributes to the relevance of the remaining jobs, allowing the system to present more useful recommendations.

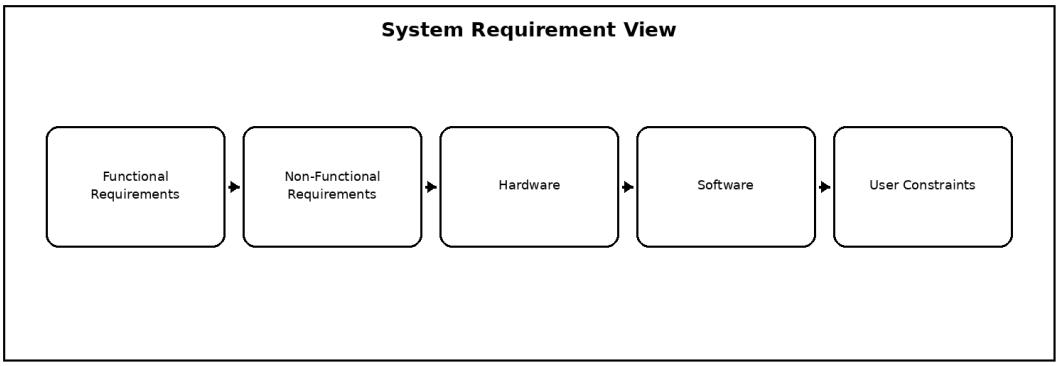


Fig. 2.6 System Requirements

3. SYSTEM DESIGN — 3.1 High-Level Architecture Diagram

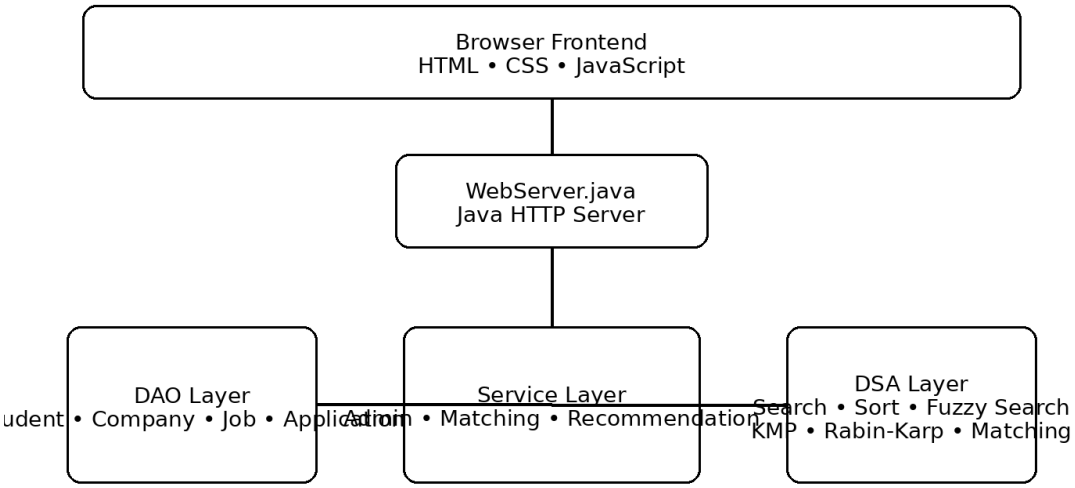


Fig. 3.1 High-Level System Architecture

The system uses a layered architecture consisting of a browser frontend, Java HTTP server, DAO layer, service layer, model layer, and DSA layer. Each layer has a defined responsibility.

The layered design reduces coupling between the user interface and algorithmic implementation. The WebServer layer is responsible for HTTP routing, DAO classes manage runtime records, service classes coordinate application logic, and DSA classes perform algorithmic processing. This organization supports both demonstration and maintenance.

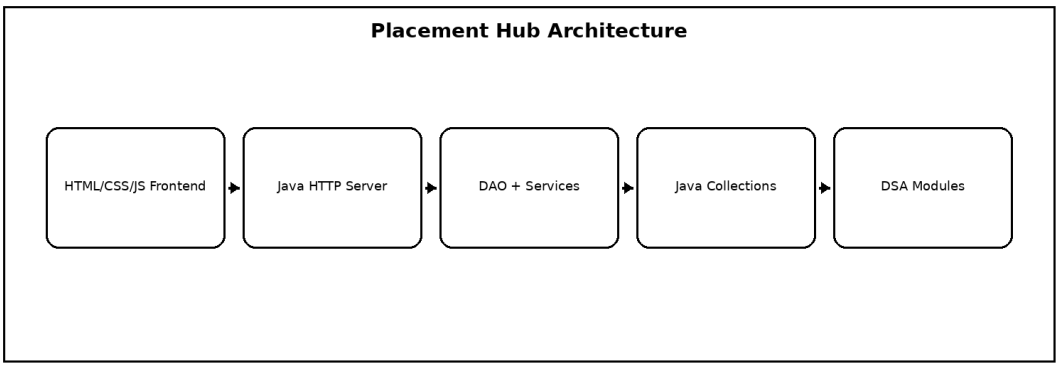


Fig. 3.2 High-Level System Architecture

3. SYSTEM DESIGN — 3.1 Architecture Rationale

The browser communicates with WebServer.java using HTTP requests. The server routes each request to the correct operation and returns structured results.

This design is suitable because the user interface remains separate from data access and algorithmic processing.

The layered design reduces coupling between the user interface and algorithmic implementation. The WebServer layer is responsible for HTTP routing, DAO classes manage runtime records, service classes coordinate application logic, and DSA classes perform algorithmic processing. This organization supports both demonstration and maintenance.

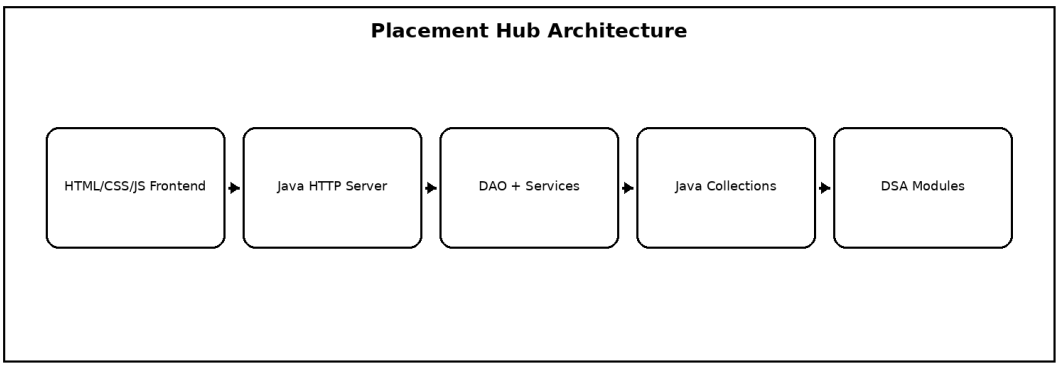


Fig. 3.3 High-Level System Architecture

3. SYSTEM DESIGN — 3.2 Presentation Layer

The presentation layer uses HTML, CSS, and JavaScript to render forms, dashboards, job cards, company information, search results, recommendations, and applications.

JavaScript manages user interaction and API requests while keeping backend implementation details outside the browser.

The API layer provides a clear contract between the browser and Java backend. Endpoints accept operation-specific inputs and return structured results. This separation allows the frontend to remain focused on presentation while the backend handles validation, data access, and DSA processing.

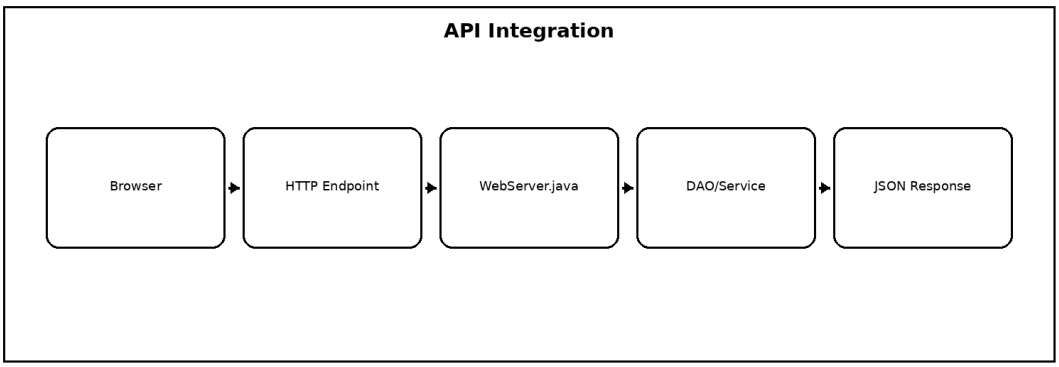


Fig. 3.4 API Integration

3. SYSTEM DESIGN — 3.2 API Layer

WebServer.java uses Java's built-in HTTP server. Endpoints are provided for health checking, login, registration, jobs, companies, search, fuzzy search, recommendations, applications, and DSA demonstrations.

POST operations are used for state-changing actions such as registration and applying for a job, while retrieval operations return available records.

The API layer provides a clear contract between the browser and Java backend. Endpoints accept operation-specific inputs and return structured results. This separation allows the frontend to remain focused on presentation while the backend handles validation, data access, and DSA processing.

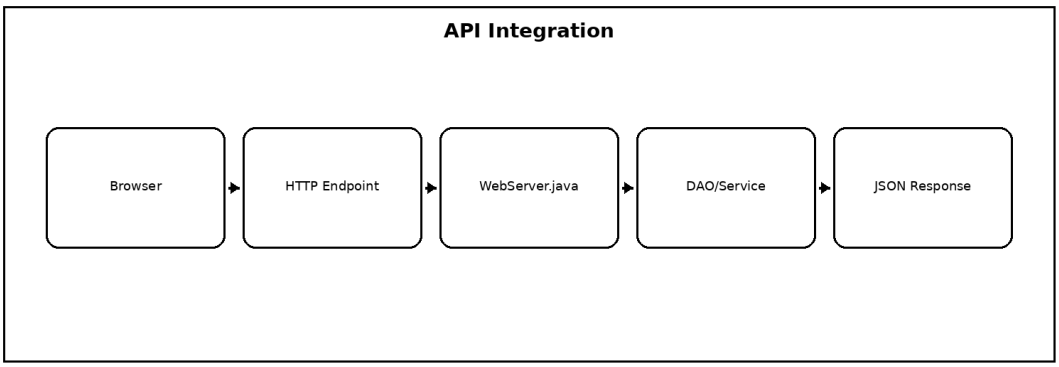


Fig. 3.5 API Integration

3. SYSTEM DESIGN — 3.2 DAO Layer

StudentDAO, CompanyDAO, JobDAO, and ApplicationDAO manage in-memory records. They provide methods for adding, retrieving, searching, and updating project entities.

The DAO abstraction isolates storage details from higher-level application logic.

Search results can be improved by considering more than exact title equality. The system can inspect relevant job fields and use fuzzy similarity so that small typing mistakes do not prevent useful results from being returned. Ranking by similarity helps place the most relevant opportunities first.

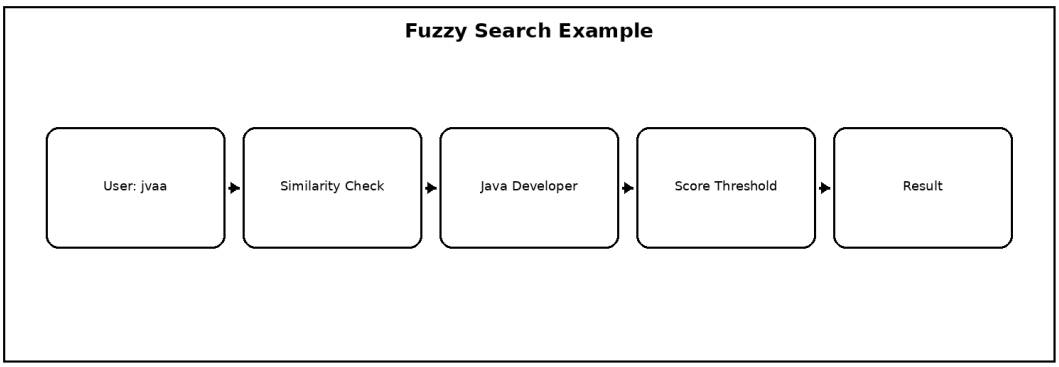


Fig. 3.6 Fuzzy Search Workflow

3. SYSTEM DESIGN — 3.2 Model Layer

Student, Company, Job, and Application classes represent the project's main entities. Models provide a consistent structure for transferring information between DAO, service, API, and frontend components.

Job and company identifiers provide the relationship needed to display company names for individual job records.

The frontend is designed around simple student actions. Forms collect input, dashboard sections expose major functions, and JavaScript calls the backend asynchronously. Results are rendered dynamically so that the user can search, inspect jobs, receive recommendations, and track applications without navigating through unrelated screens.

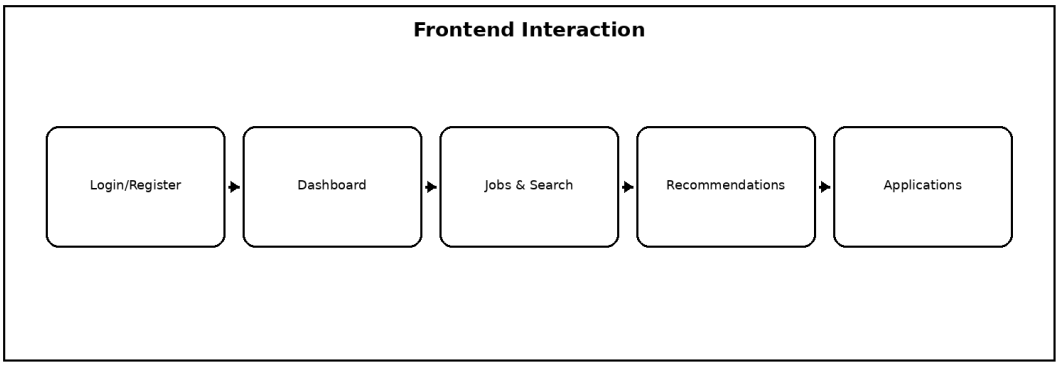


Fig. 3.7 Frontend Interaction

3. SYSTEM DESIGN — 3.3 Data Structures

ArrayList is used as the primary runtime storage structure. HashMap supports key-value representations. Stack and Queue demonstrate linear data-structure operations, while Trie and graph concepts support specialized DSA demonstrations.

The choice of ArrayList is appropriate for the demonstration dataset because the system frequently iterates over the available records.

The choice of data structure depends on the operation. ArrayList is suitable for maintaining ordered collections of students, companies, jobs, and applications. HashMap supports key-value style access, while Stack and Queue demonstrate LIFO and FIFO processing. Trie and graph structures support more specialized algorithmic demonstrations.

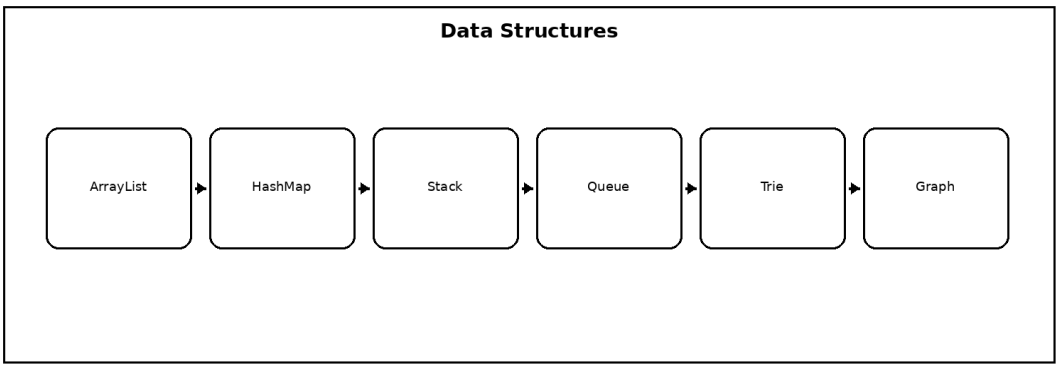


Fig. 3.8 Data Structures Used

3. SYSTEM DESIGN — 3.3 Algorithms

KMP and Rabin-Karp demonstrate pattern matching. Edit distance and Damerau-Levenshtein support similarity calculations. Sorting organizes job results, while matching and recommendation compare student attributes with job requirements.

Dynamic programming concepts are used in string and optimization-related demonstrations. Bipartite matching and maximum-flow ideas are included as advanced placement matching concepts.

The DSA modules are not isolated textbook exercises; they are connected to placement operations. Pattern matching supports efficient text search, fuzzy methods tolerate spelling variation, sorting organizes results, and matching/recommendation logic combines multiple student and job attributes.

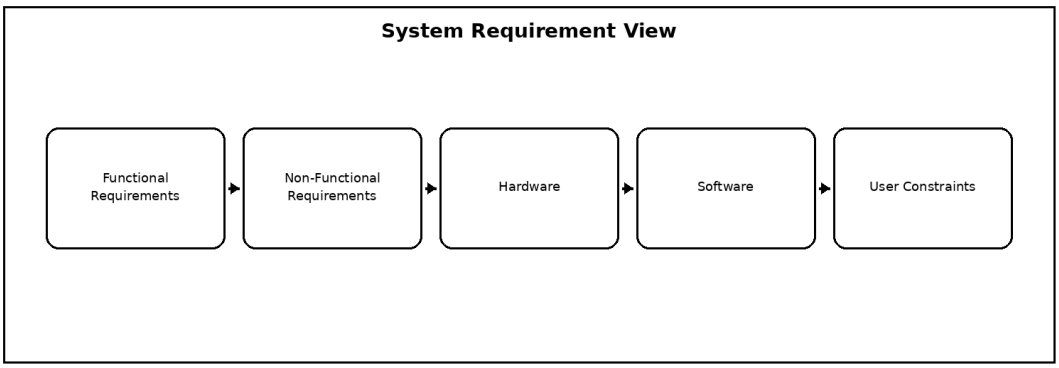


Fig. 3.9 System Requirements

3. SYSTEM DESIGN — 3.3 Fuzzy Search

Fuzzy search accepts a query and a similarity threshold. Each job is compared against searchable text such as role and relevant fields. Results meeting the threshold are returned with a similarity score.

The threshold is normalized consistently so that percentage values can be used by the user interface.

Search results can be improved by considering more than exact title equality. The system can inspect relevant job fields and use fuzzy similarity so that small typing mistakes do not prevent useful results from being returned. Ranking by similarity helps place the most relevant opportunities first.

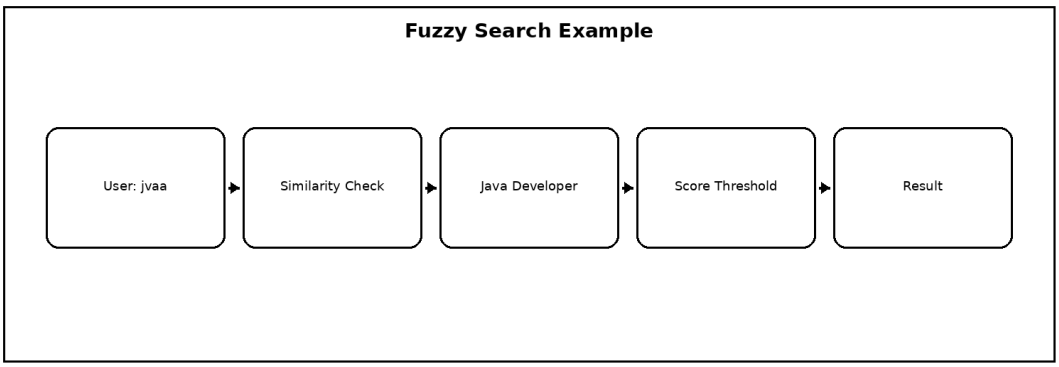


Fig. 3.10 Fuzzy Search Workflow

3. SYSTEM DESIGN — 3.4 Data Flow

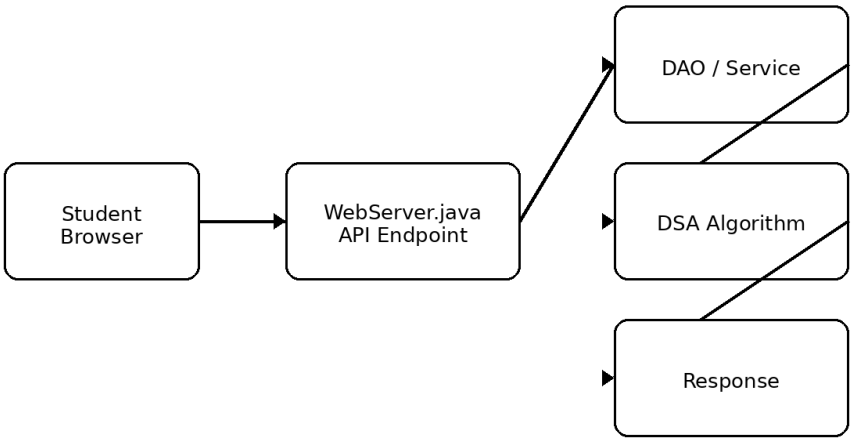


Fig. 3.11 Recommendation Workflow

A user action begins in the browser. The frontend sends an HTTP request, WebServer.java parses it, and the required DAO, service, or DSA module is invoked.

The processed result is converted into a response and displayed by the frontend.

For each operation, information travels from the user interface to the API endpoint, is validated and processed by the backend, and then returns as a response that the frontend renders. Keeping this path explicit makes debugging easier because input, processing, and output can be inspected separately.

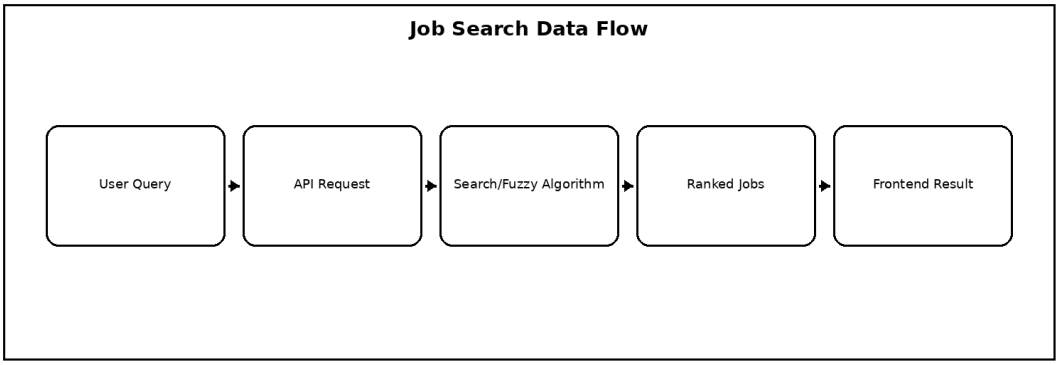


Fig. 3.12 Job Search Data Flow

3. SYSTEM DESIGN — 3.4 Job Matching Workflow

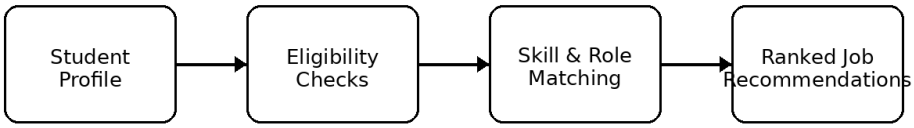


Fig. 3.13 Student-to-Job Matching Workflow

Eligibility is treated as a constraint-checking operation. Minimum CGPA, branch, backlog count, and skills are evaluated before a job is considered fully suitable.

Recommendation can then rank jobs according to compatibility, making the distinction between hard constraints and ranking preferences clear.

The matching process can be viewed as a sequence of filters followed by ranking. Hard constraints such as branch, CGPA, and backlog limits remove unsuitable opportunities first. Skill similarity then contributes to the relevance of the remaining jobs, allowing the system to present more useful recommendations.

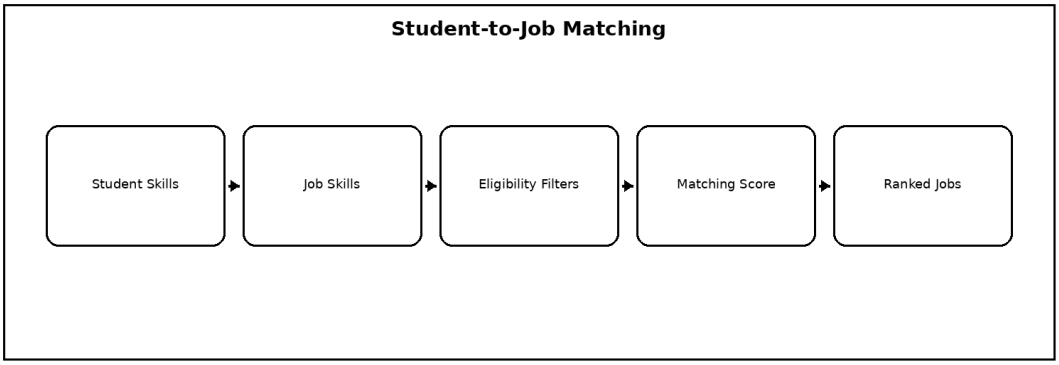


Fig. 3.14 Student-to-Job Matching Workflow

3. SYSTEM DESIGN — 3.4 Application Data Flow

When a student applies, the system verifies that the job exists and checks for duplicate applications. An Application object is then created with an ID, student ID, job ID, date, and status.

The application can later be combined with job and company data to present meaningful information to the student.

For each operation, information travels from the user interface to the API endpoint, is validated and processed by the backend, and then returns as a response that the frontend renders. Keeping this path explicit makes debugging easier because input, processing, and output can be inspected separately.

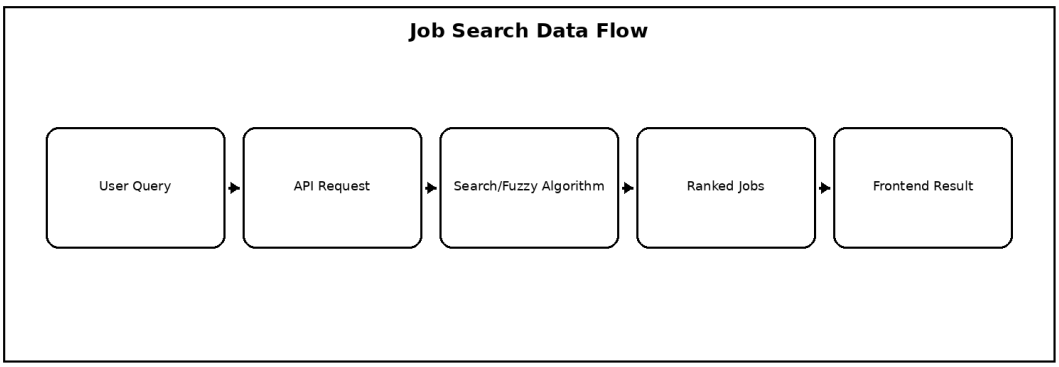


Fig. 3.15 Job Search Data Flow

4. IMPLEMENTATION — 4.1 Module-Wise Implementation

The implementation is divided into authentication, student, company, job, search, recommendation, eligibility, application, administration, and DSA modules. This modular organization keeps the project understandable and testable.

Implementation is divided into small, testable units. Model classes represent data, DAO classes provide collection-based access, service classes coordinate higher-level operations, and DSA classes contain algorithmic logic. This modular structure also makes it easier to locate and correct errors during development.

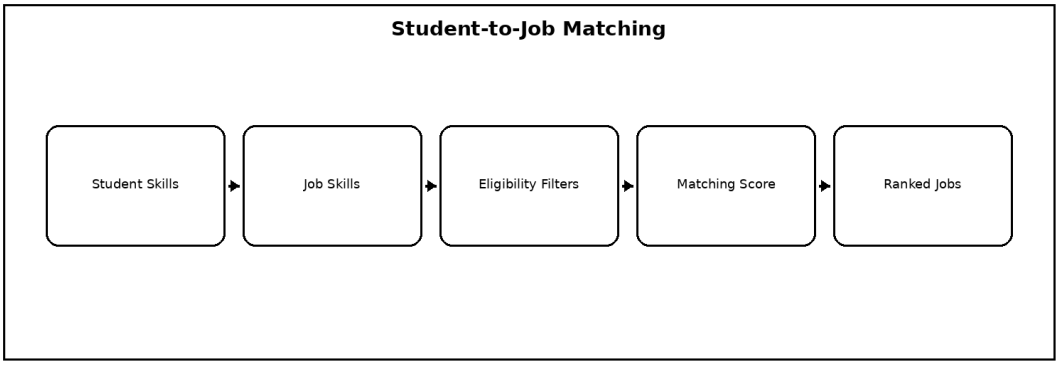


Fig. 4.1 Student-to-Job Matching Workflow

4. IMPLEMENTATION — Student Module

Student objects contain identity, authentication, academic, and skill information. StudentDAO supports registration, login, email lookup, and retrieval of all students.

Implementation is divided into small, testable units. Model classes represent data, DAO classes provide collection-based access, service classes coordinate higher-level operations, and DSA classes contain algorithmic logic. This modular structure also makes it easier to locate and correct errors during development.

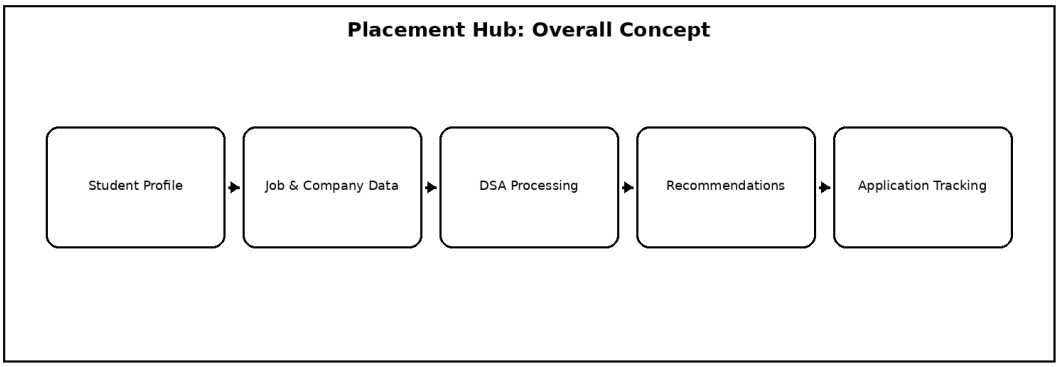


Fig. 4.2 Placement Hub Overview

4. IMPLEMENTATION — Company Module

Company objects contain company ID, name, location, website, and description. CompanyDAO initializes the 25-company demonstration dataset and provides retrieval and addition operations.

Implementation is divided into small, testable units. Model classes represent data, DAO classes provide collection-based access, service classes coordinate higher-level operations, and DSA classes contain algorithmic logic. This modular structure also makes it easier to locate and correct errors during development.

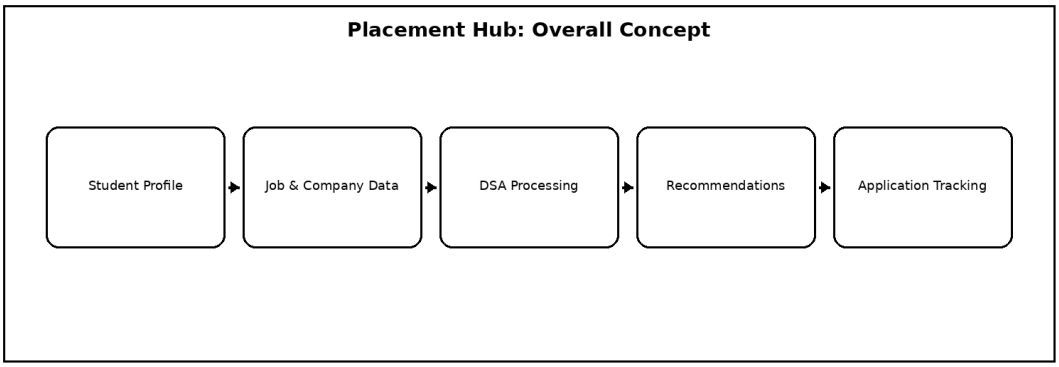


Fig. 4.3 Placement Hub Overview

4. IMPLEMENTATION — Job Module

Job objects contain job ID, company ID, role, package, minimum CGPA, branches, skills, maximum backlogs, drive date, and description. JobDAO initializes the 50-job dataset.

Implementation is divided into small, testable units. Model classes represent data, DAO classes provide collection-based access, service classes coordinate higher-level operations, and DSA classes contain algorithmic logic. This modular structure also makes it easier to locate and correct errors during development.

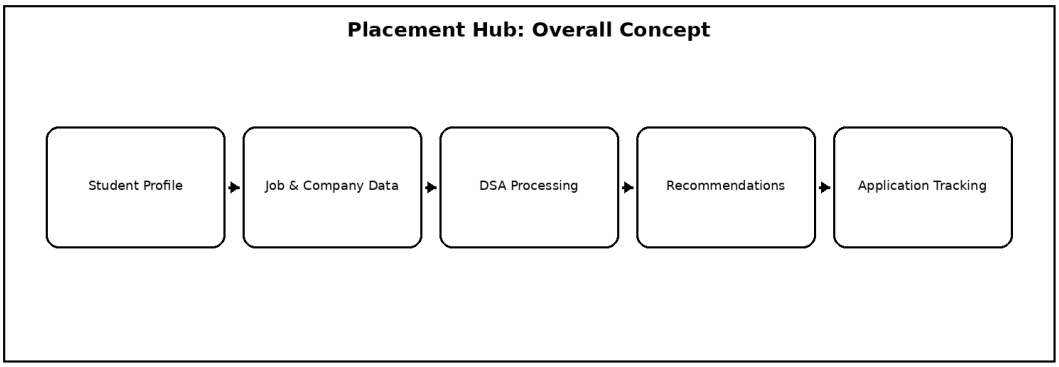


Fig. 4.4 Placement Hub Overview

4. IMPLEMENTATION — Application Module

ApplicationDAO stores application records in an ArrayList. It checks duplicates, creates new records, assigns IDs, stores the date, and initializes the application status.

Implementation is divided into small, testable units. Model classes represent data, DAO classes provide collection-based access, service classes coordinate higher-level operations, and DSA classes contain algorithmic logic. This modular structure also makes it easier to locate and correct errors during development.

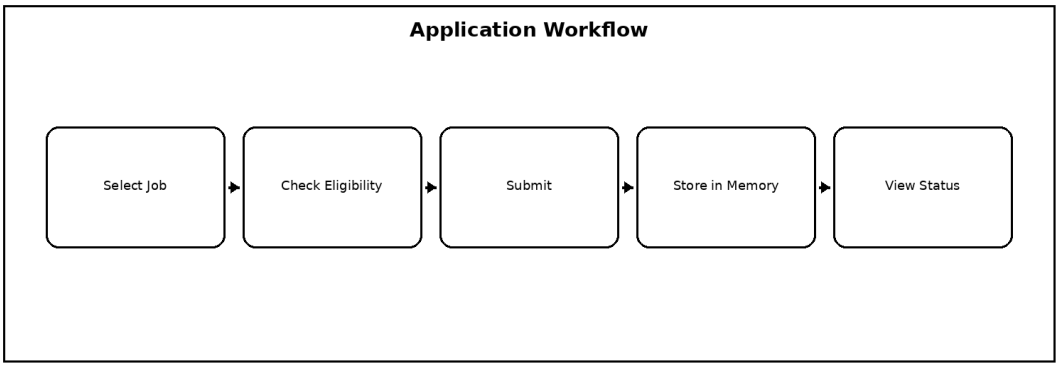


Fig. 4.5 Application Workflow

4. IMPLEMENTATION — 4.2 Front-End Logic

The frontend provides registration, login, dashboard, jobs, companies, search, recommendations, eligibility, and application views. JavaScript fetches API responses and updates the visible interface.

The frontend is designed around simple student actions. Forms collect input, dashboard sections expose major functions, and JavaScript calls the backend asynchronously. Results are rendered dynamically so that the user can search, inspect jobs, receive recommendations, and track applications without navigating through unrelated screens.

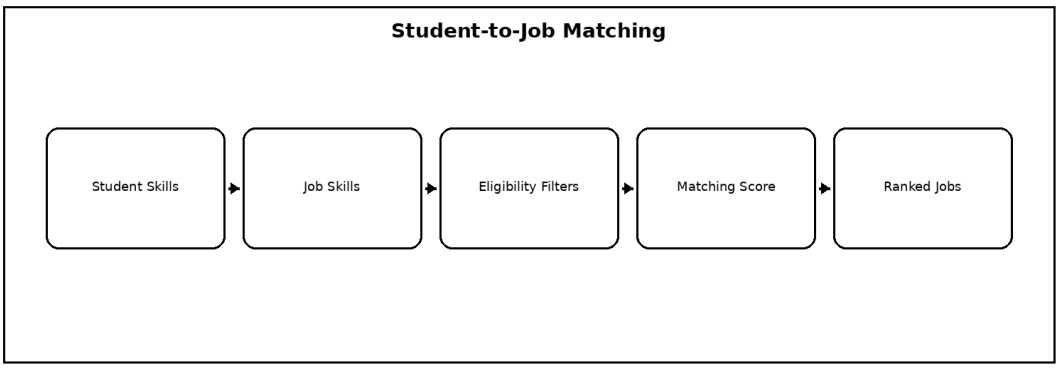


Fig. 4.6 Student-to-Job Matching Workflow

4. IMPLEMENTATION — Dashboard

The dashboard provides the main navigation path. Students can move from authentication to profile, jobs, companies, recommendations, eligibility, and applications.

Implementation is divided into small, testable units. Model classes represent data, DAO classes provide collection-based access, service classes coordinate higher-level operations, and DSA classes contain algorithmic logic. This modular structure also makes it easier to locate and correct errors during development.

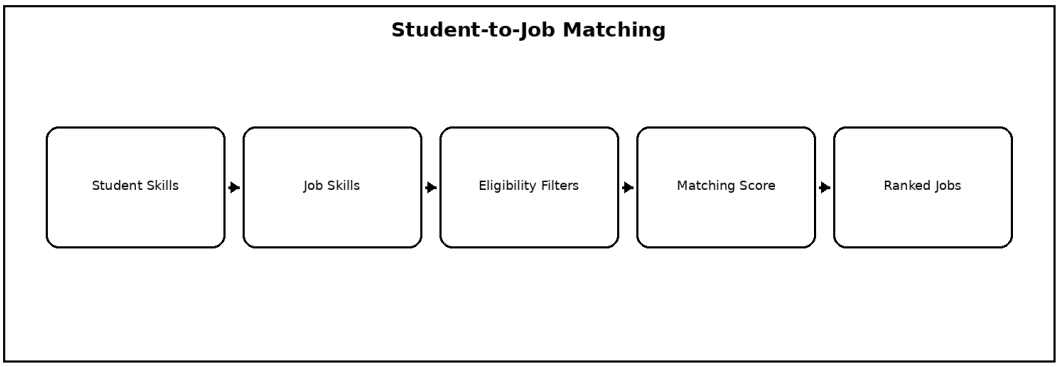


Fig. 4.7 Student-to-Job Matching Workflow

4. IMPLEMENTATION — Job Search Interface

The normal search interface sends a keyword to the search endpoint and renders returned jobs. Fuzzy search provides typo tolerance when exact matching is not sufficient.

The matching process can be viewed as a sequence of filters followed by ranking. Hard constraints such as branch, CGPA, and backlog limits remove unsuitable opportunities first. Skill similarity then contributes to the relevance of the remaining jobs, allowing the system to present more useful recommendations.

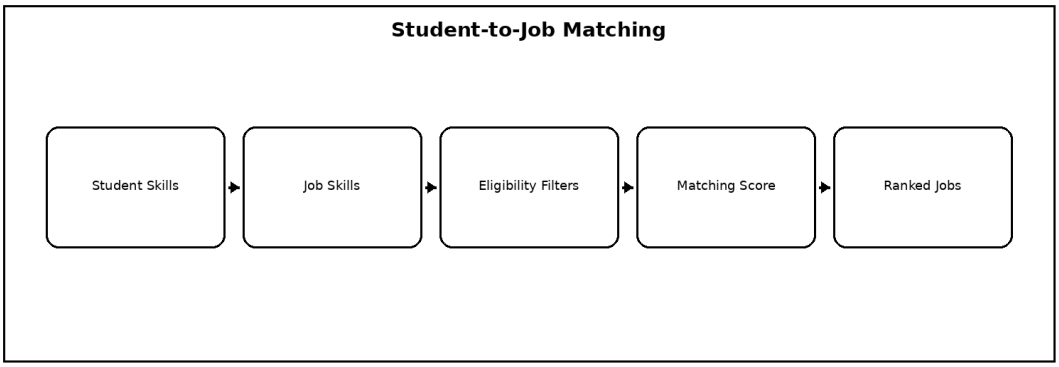


Fig. 4.8 Student-to-Job Matching Workflow

4. IMPLEMENTATION — 4.3 API Integration

WebServer.java registers HTTP contexts for the major application operations. It receives requests, parses parameters, invokes project components, and sends responses to the browser.

The API layer provides a clear contract between the browser and Java backend. Endpoints accept operation-specific inputs and return structured results. This separation allows the frontend to remain focused on presentation while the backend handles validation, data access, and DSA processing.

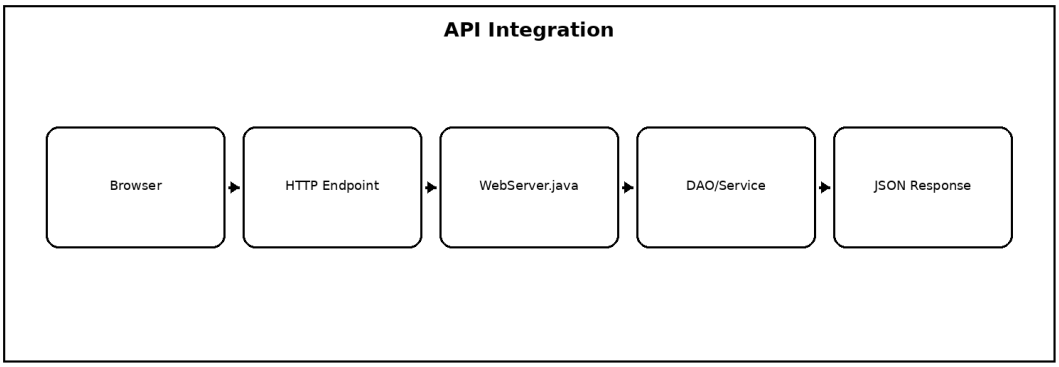


Fig. 4.9 API Integration

4. IMPLEMENTATION — API Validation

The API validates inputs and handles missing students, missing jobs, invalid credentials, duplicate applications, and other invalid states. This prevents the frontend from directly manipulating internal collections.

The frontend is designed around simple student actions. Forms collect input, dashboard sections expose major functions, and JavaScript calls the backend asynchronously. Results are rendered dynamically so that the user can search, inspect jobs, receive recommendations, and track applications without navigating through unrelated screens.

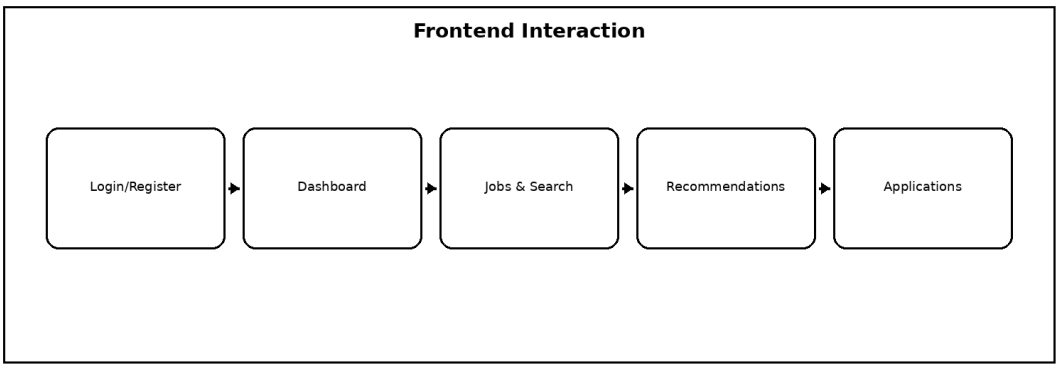


Fig. 4.10 Frontend Interaction

4. IMPLEMENTATION — 4.4 DSA Implementation

DSA classes provide reusable implementations for searching, sorting, fuzzy matching, pattern matching, and recommendation. These classes are invoked by application workflows.

The matching process can be viewed as a sequence of filters followed by ranking. Hard constraints such as branch, CGPA, and backlog limits remove unsuitable opportunities first. Skill similarity then contributes to the relevance of the remaining jobs, allowing the system to present more useful recommendations.

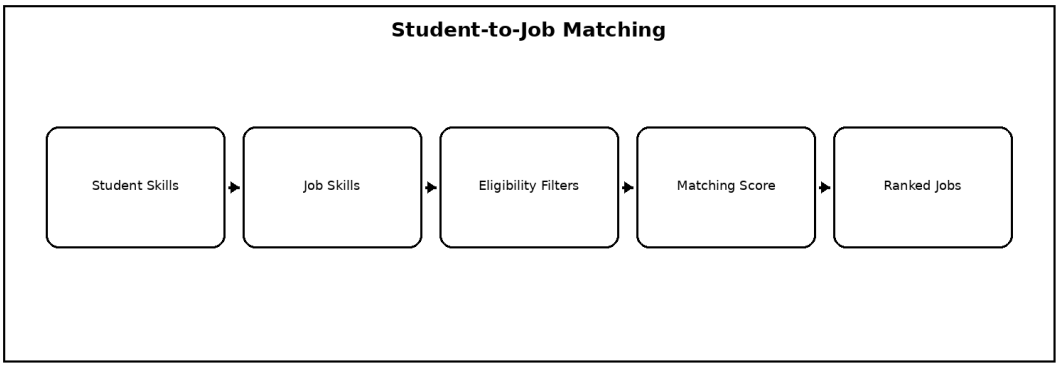


Fig. 4.11 Student-to-Job Matching Workflow

4. IMPLEMENTATION — KMP

KMP uses a longest-prefix-suffix array to avoid restarting the pattern comparison after a mismatch. It provides an efficient pattern-searching demonstration for job-related text.

Implementation is divided into small, testable units. Model classes represent data, DAO classes provide collection-based access, service classes coordinate higher-level operations, and DSA classes contain algorithmic logic. This modular structure also makes it easier to locate and correct errors during development.

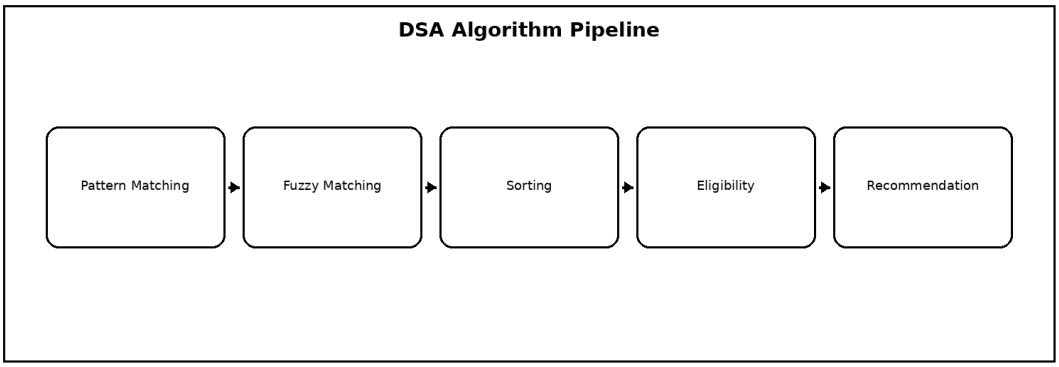


Fig. 4.12 DSA Algorithm Pipeline

4. IMPLEMENTATION — Rabin-Karp

Rabin-Karp uses a rolling hash to locate candidate pattern positions and then verifies matching text. It demonstrates an alternative approach to pattern searching.

The matching process can be viewed as a sequence of filters followed by ranking. Hard constraints such as branch, CGPA, and backlog limits remove unsuitable opportunities first. Skill similarity then contributes to the relevance of the remaining jobs, allowing the system to present more useful recommendations.

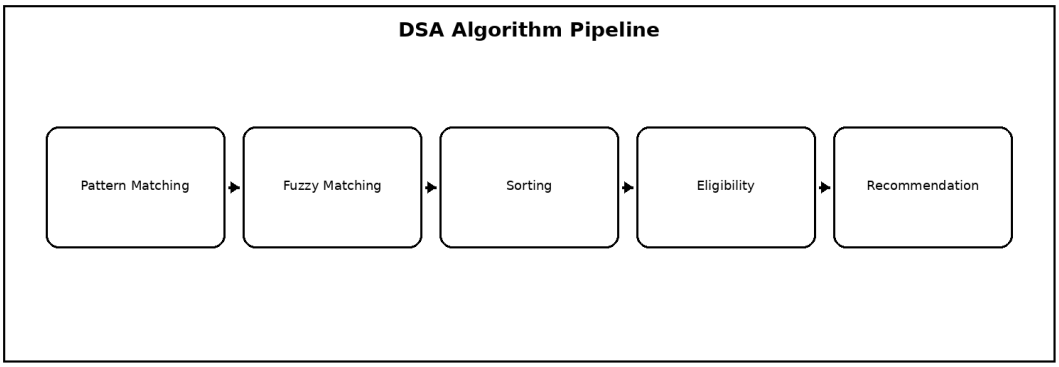


Fig. 4.13 DSA Algorithm Pipeline

4. IMPLEMENTATION — Edit Distance

Edit distance calculates the minimum insertions, deletions, and substitutions needed to transform one string into another. It provides a basis for similarity-based job search.

Implementation is divided into small, testable units. Model classes represent data, DAO classes provide collection-based access, service classes coordinate higher-level operations, and DSA classes contain algorithmic logic. This modular structure also makes it easier to locate and correct errors during development.

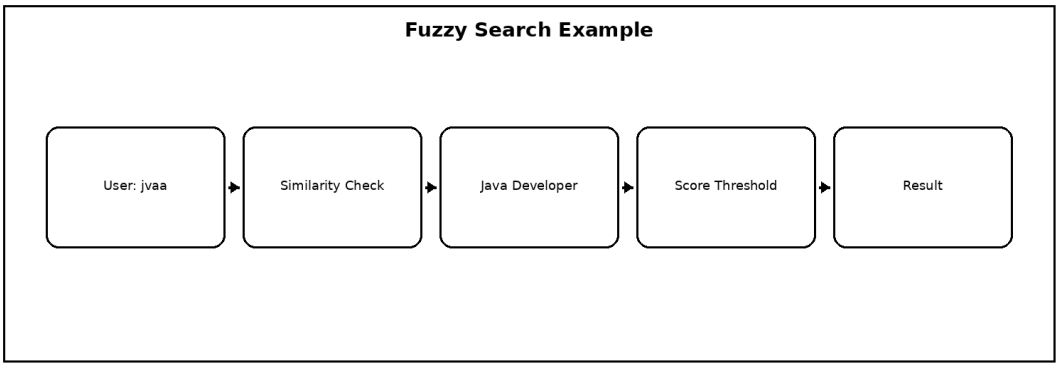


Fig. 4.14 Fuzzy Search Workflow

4. IMPLEMENTATION — Recommendation

Recommendation evaluates student and job compatibility. Branch and academic requirements are checked first, while skill overlap contributes to the ranking of suitable jobs.

Implementation is divided into small, testable units. Model classes represent data, DAO classes provide collection-based access, service classes coordinate higher-level operations, and DSA classes contain algorithmic logic. This modular structure also makes it easier to locate and correct errors during development.

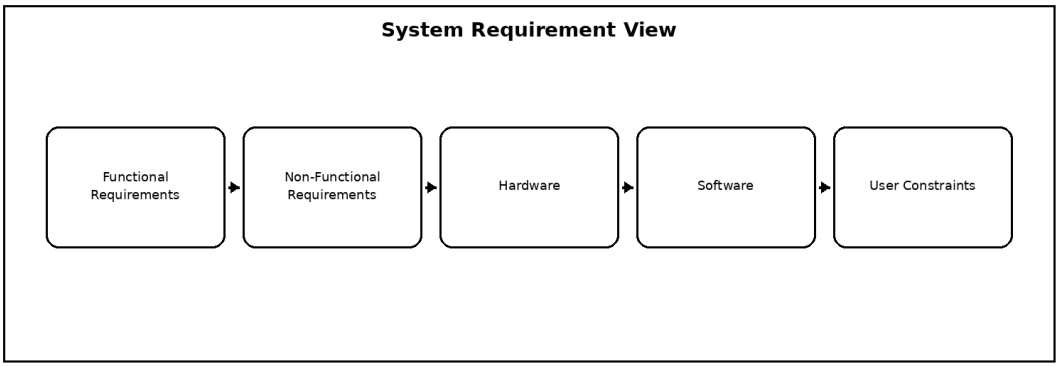


Fig. 4.15 System Requirements

4. IMPLEMENTATION — Pseudocode

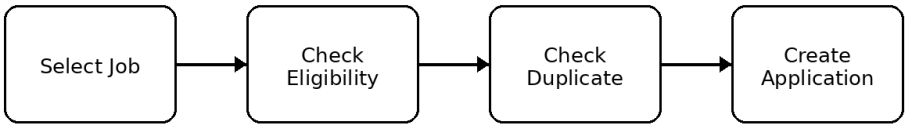


Fig. 4.16 Application Workflow

Search pseudocode: read the query; retrieve jobs; compare the query with searchable fields; store matching records; rank or sort results; return the final list.

The matching process can be viewed as a sequence of filters followed by ranking. Hard constraints such as branch, CGPA, and backlog limits remove unsuitable opportunities first. Skill similarity then contributes to the relevance of the remaining jobs, allowing the system to present more useful recommendations.

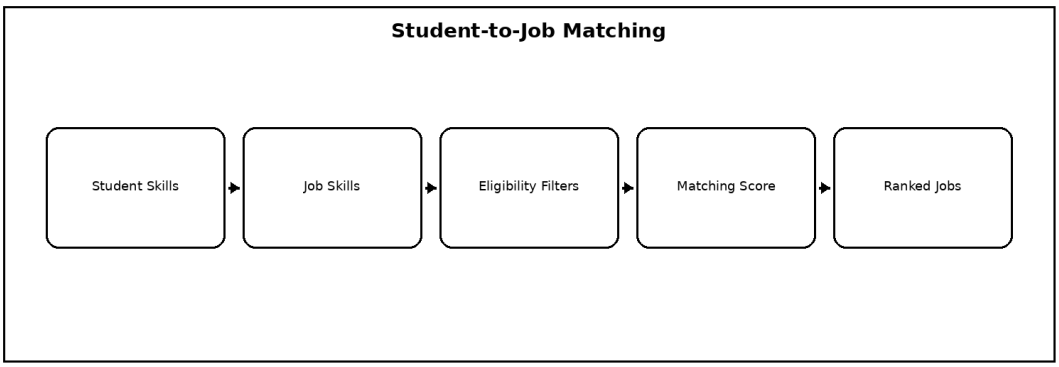


Fig. 4.17 Student-to-Job Matching Workflow

4. IMPLEMENTATION — Pseudocode

Fuzzy-search pseudocode: read query and threshold; retrieve jobs; calculate similarity; retain jobs meeting the threshold; sort by similarity; return results.

Implementation is divided into small, testable units. Model classes represent data, DAO classes provide collection-based access, service classes coordinate higher-level operations, and DSA classes contain algorithmic logic. This modular structure also makes it easier to locate and correct errors during development.

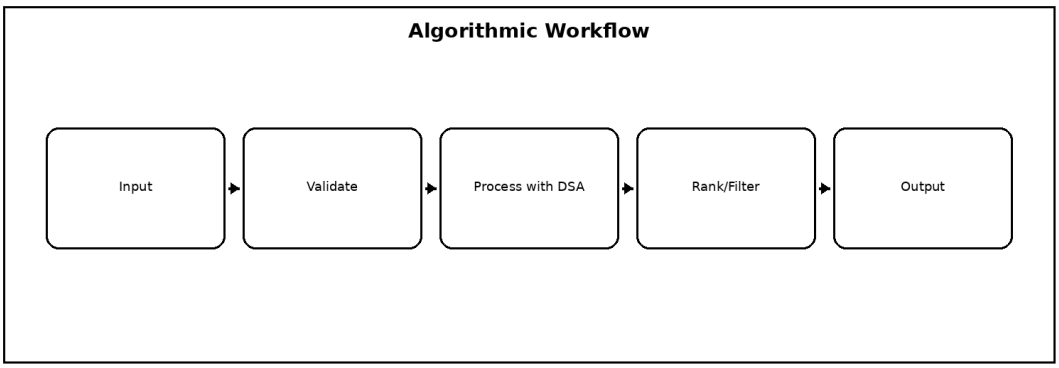


Fig. 4.18 Algorithmic Workflow

4. IMPLEMENTATION — Pseudocode

Eligibility pseudocode: read student and job; compare CGPA, branch, backlogs, and skills; return eligible or ineligible with reasons. Application pseudocode then verifies the job, checks duplicates, creates an application, and stores it.

Implementation is divided into small, testable units. Model classes represent data, DAO classes provide collection-based access, service classes coordinate higher-level operations, and DSA classes contain algorithmic logic. This modular structure also makes it easier to locate and correct errors during development.

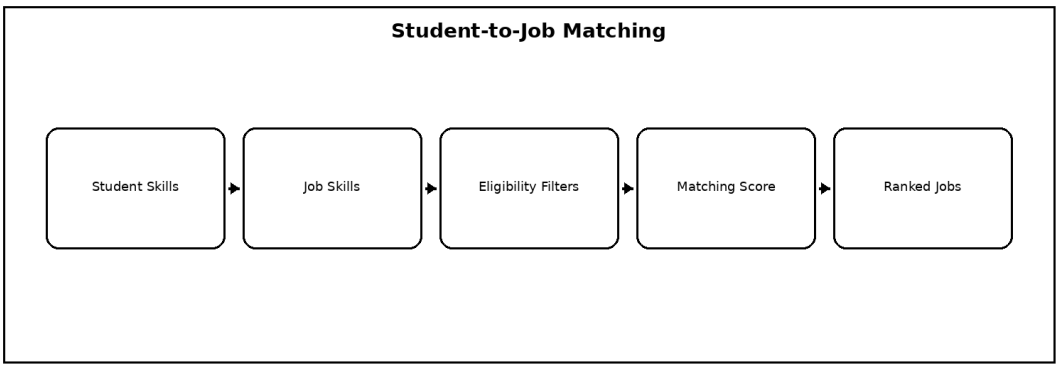


Fig. 4.19 Student-to-Job Matching Workflow

4. IMPLEMENTATION — Testing and Integration

Testing was performed at module and workflow level. Maven compilation was used after major changes, followed by browser-based checks of login, jobs, search, fuzzy search, recommendation, eligibility, and applications.

Implementation is divided into small, testable units. Model classes represent data, DAO classes provide collection-based access, service classes coordinate higher-level operations, and DSA classes contain algorithmic logic. This modular structure also makes it easier to locate and correct errors during development.

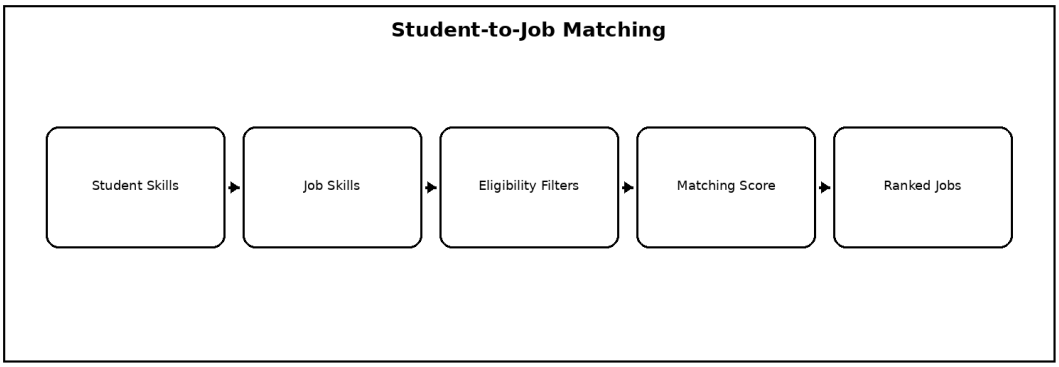


Fig. 4.20 Student-to-Job Matching Workflow

5. RESULTS — 5.1 Functional Testing

Functional testing verified the main student and administrative operations. Valid registration, login, job viewing, company viewing, search, recommendation, eligibility, and application operations were checked.

Functional requirements describe what the system must do from the user's perspective. They form the basis for API endpoints and frontend actions. Each function is connected to a backend operation so that the final application can be tested from input through processing to visible output.

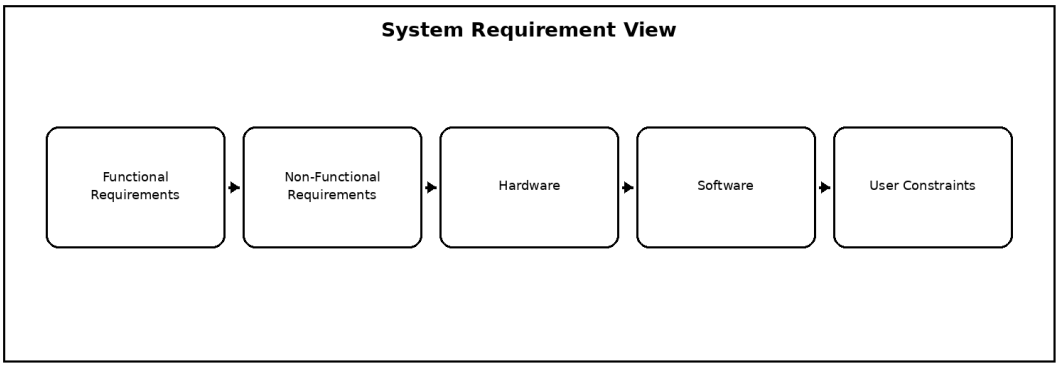


Fig. 5.1 System Requirements

5. RESULTS — Registration and Login

Valid registration creates a student record with a unique ID. Duplicate email registration is rejected. Valid credentials allow login while invalid credentials do not create a successful session.

The implemented workflow demonstrates that the project can manage student information, company records, job opportunities, search requests, recommendations, eligibility checks, and applications in one runtime session. The results section documents these outcomes through functional test cases and workflow descriptions.

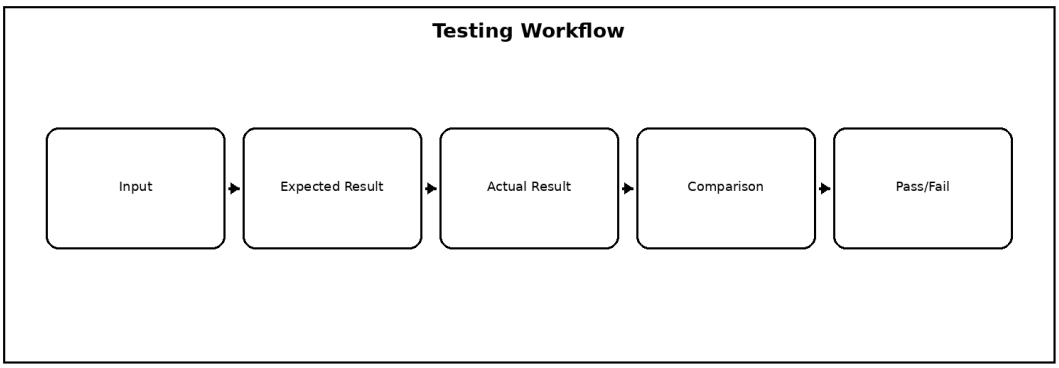


Fig. 5.2 Testing Workflow

5. RESULTS — Companies and Jobs

The company module displays 25 companies. The job module displays 50 job opportunities containing role, package, eligibility information, skills, and drive date.

The implemented workflow demonstrates that the project can manage student information, company records, job opportunities, search requests, recommendations, eligibility checks, and applications in one runtime session. The results section documents these outcomes through functional test cases and workflow descriptions.

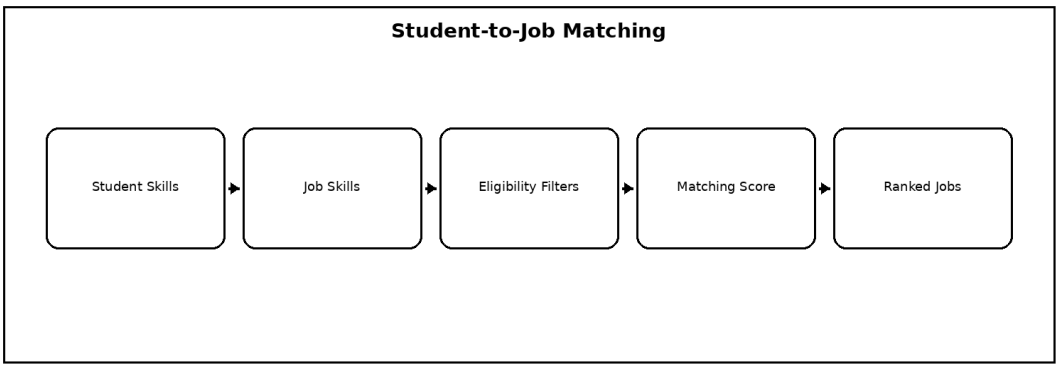


Fig. 5.3 Student-to-Job Matching Workflow

5. RESULTS — Normal Search

Normal search retrieves jobs associated with the entered keyword. It reduces the visible result set and provides a direct way for students to find roles of interest.

Search results can be improved by considering more than exact title equality. The system can inspect relevant job fields and use fuzzy similarity so that small typing mistakes do not prevent useful results from being returned. Ranking by similarity helps place the most relevant opportunities first.

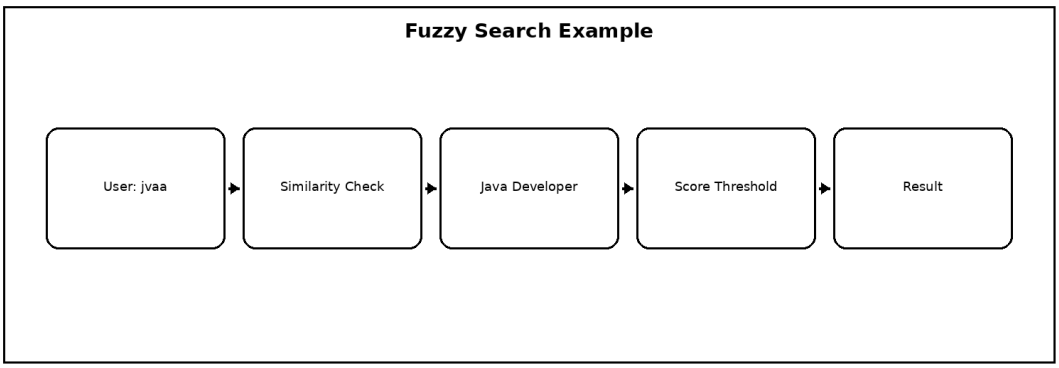


Fig. 5.4 Fuzzy Search Workflow

5. RESULTS — Fuzzy Search

Fuzzy search was designed to tolerate spelling errors. For example, a query similar to “jvaa” can return Java-related opportunities when its similarity score satisfies the configured threshold.

Search results can be improved by considering more than exact title equality. The system can inspect relevant job fields and use fuzzy similarity so that small typing mistakes do not prevent useful results from being returned. Ranking by similarity helps place the most relevant opportunities first.

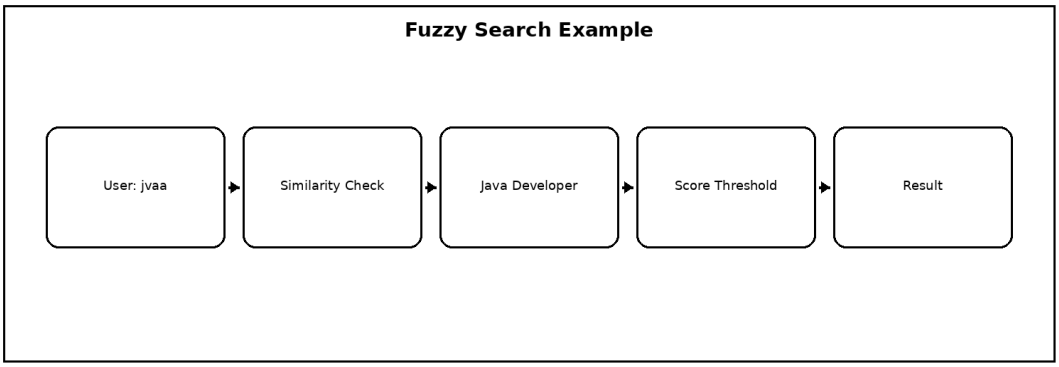


Fig. 5.5 Fuzzy Search Workflow

5. RESULTS — Sorting

Sorting helps students compare opportunities by useful attributes such as package, role, or relevance. This becomes increasingly useful as the job dataset grows.

The implemented workflow demonstrates that the project can manage student information, company records, job opportunities, search requests, recommendations, eligibility checks, and applications in one runtime session. The results section documents these outcomes through functional test cases and workflow descriptions.

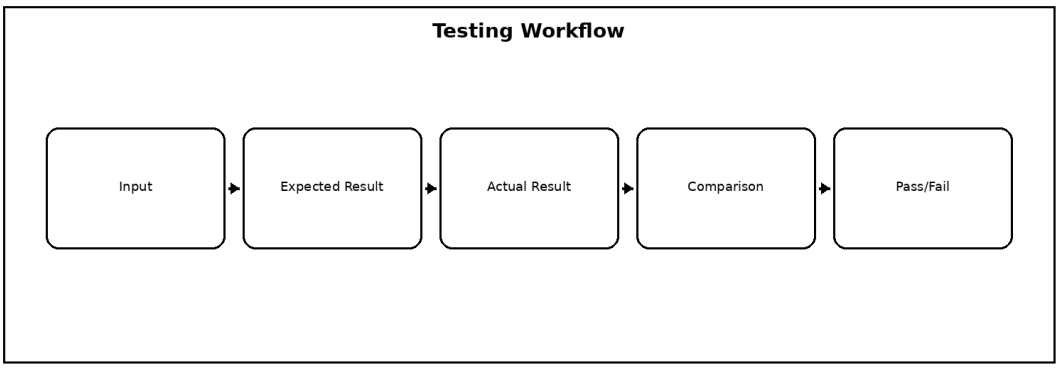


Fig. 5.6 Testing Workflow

5. RESULTS — Eligibility

Eligibility testing checks minimum CGPA, allowed branch, maximum backlogs, and required skills. The result provides a meaningful decision before application.

Testing was considered at both feature and integration levels. Individual operations such as registration and search were checked first, followed by complete workflows such as login to recommendation and eligibility to application. The objective was to verify not only that each endpoint responds, but also that connected modules exchange the expected information.

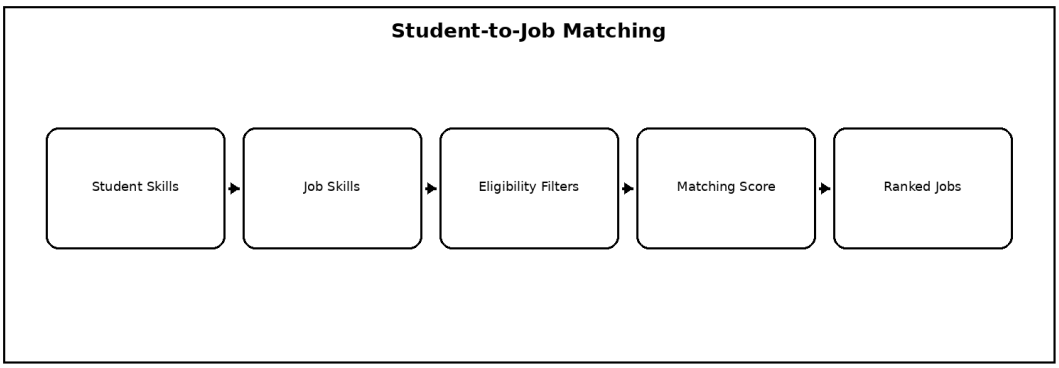


Fig. 5.7 Student-to-Job Matching Workflow

5. RESULTS — Recommendations

Recommendation ranks suitable jobs using compatibility information. A profile containing Java and DSA skills can receive stronger matches for Java and backend roles, while AI/ML skills can match machine-learning opportunities.

Recommendations are generated from the relationship between a student's profile and the available jobs. Academic constraints are considered before relevance is ranked. This approach avoids recommending opportunities that are clearly outside the student's eligibility and focuses the final list on actionable choices.

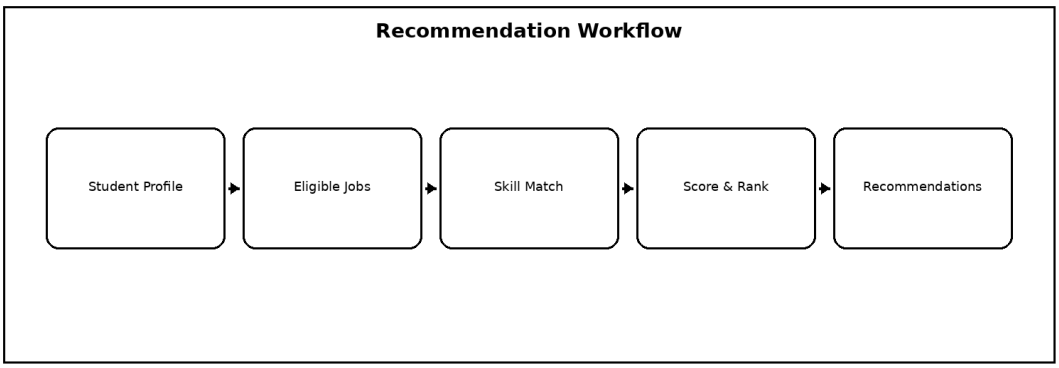


Fig. 5.8 Recommendation Workflow

5. RESULTS — Application Submission

A valid application creates an Application object with application ID, student ID, job ID, date, and initial status. Duplicate applications are prevented.

The implemented workflow demonstrates that the project can manage student information, company records, job opportunities, search requests, recommendations, eligibility checks, and applications in one runtime session. The results section documents these outcomes through functional test cases and workflow descriptions.

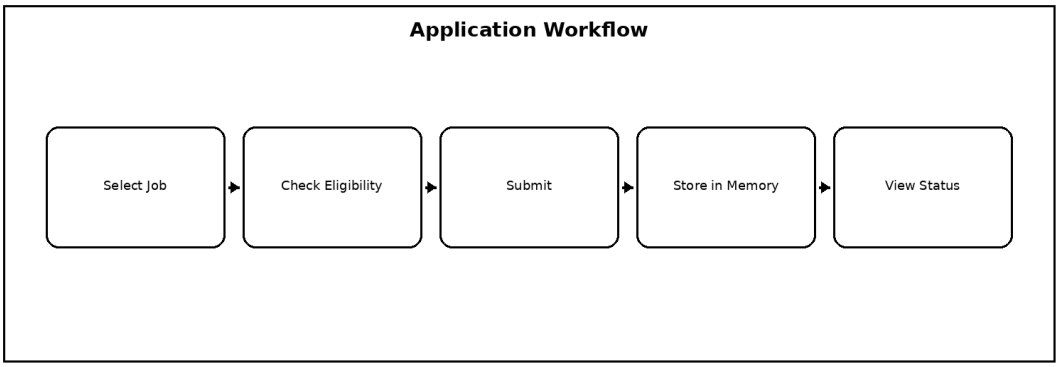


Fig. 5.9 Application Workflow

5. RESULTS — Application Viewing

Application views combine application information with job and company details. This allows students to understand which company and role each application represents.

The implemented workflow demonstrates that the project can manage student information, company records, job opportunities, search requests, recommendations, eligibility checks, and applications in one runtime session. The results section documents these outcomes through functional test cases and workflow descriptions.

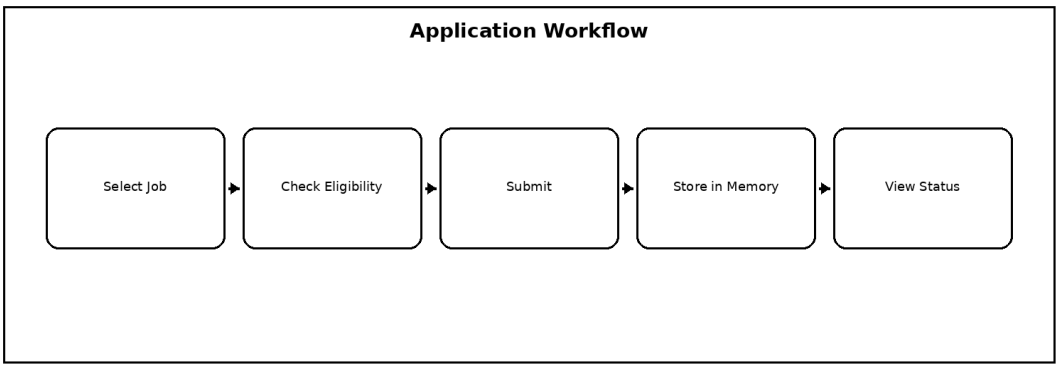


Fig. 5.10 Application Workflow

5. RESULTS — Admin Operations

Administrative views can display students, companies, jobs, and applications. Application status can be updated through the service and DAO layers.

The implemented workflow demonstrates that the project can manage student information, company records, job opportunities, search requests, recommendations, eligibility checks, and applications in one runtime session. The results section documents these outcomes through functional test cases and workflow descriptions.

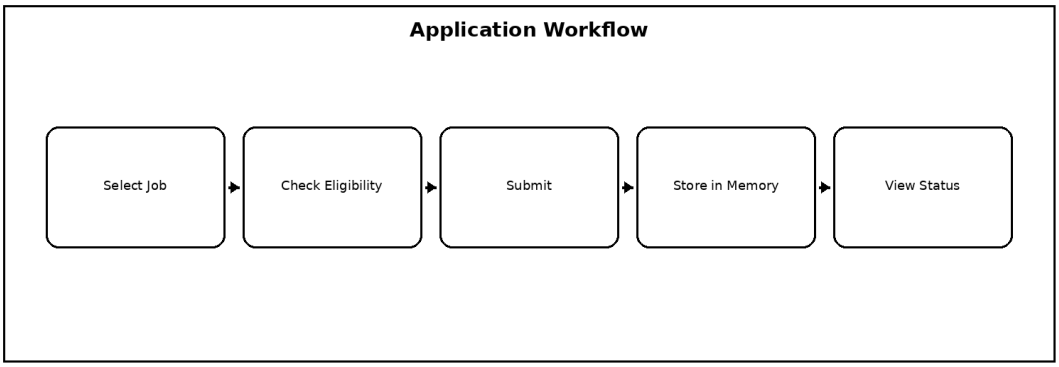


Fig. 5.11 Application Workflow

5. RESULTS — DSA Demonstration

The main program can demonstrate KMP, Rabin-Karp, and edit distance. This provides a direct academic demonstration of core DSA functionality.

Functional requirements describe what the system must do from the user's perspective. They form the basis for API endpoints and frontend actions. Each function is connected to a backend operation so that the final application can be tested from input through processing to visible output.

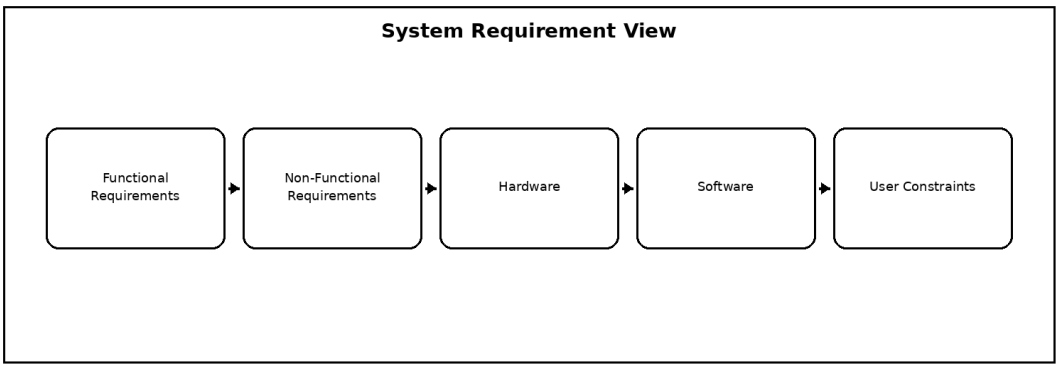


Fig. 5.12 System Requirements

5. RESULTS — Test Matrix

Representative tests include valid and invalid login, duplicate registration, empty search, fuzzy search with typos, eligible and ineligible jobs, duplicate applications, and application retrieval.

Search results can be improved by considering more than exact title equality. The system can inspect relevant job fields and use fuzzy similarity so that small typing mistakes do not prevent useful results from being returned. Ranking by similarity helps place the most relevant opportunities first.

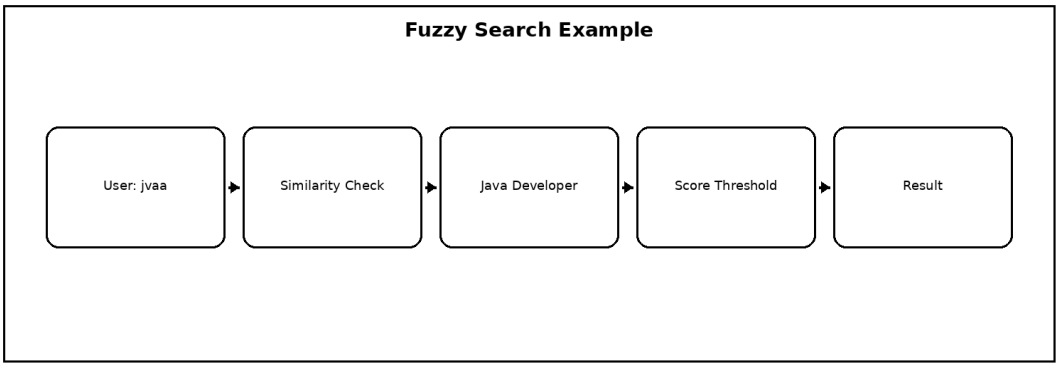


Fig. 5.13 Fuzzy Search Workflow

5. RESULTS — Performance Observation

For the demonstration dataset, in-memory processing is responsive. The main performance factors are the number of records and the complexity of the search, similarity, and matching algorithms.

The DSA modules are not isolated textbook exercises; they are connected to placement operations. Pattern matching supports efficient text search, fuzzy methods tolerate spelling variation, sorting organizes results, and matching/recommendation logic combines multiple student and job attributes.

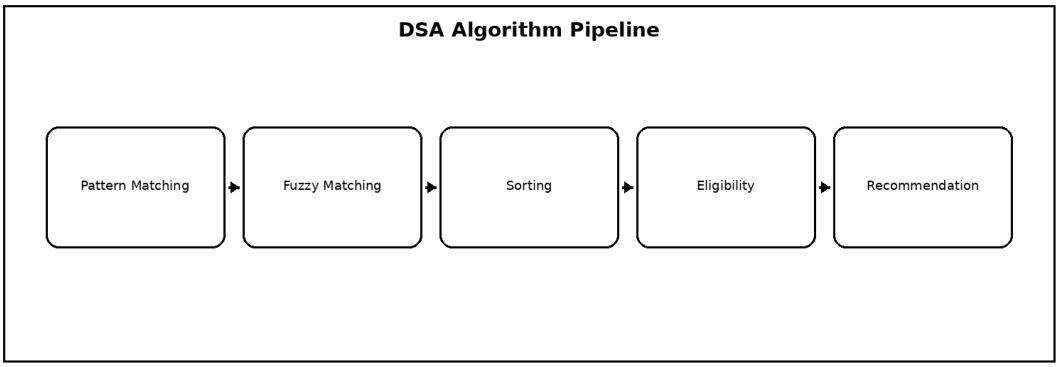


Fig. 5.14 DSA Algorithm Pipeline

5. RESULTS — Reliability Observation

Layer separation and validation reduce common invalid states. DAO methods provide controlled access to project records while services combine related operations.

The implemented workflow demonstrates that the project can manage student information, company records, job opportunities, search requests, recommendations, eligibility checks, and applications in one runtime session. The results section documents these outcomes through functional test cases and workflow descriptions.

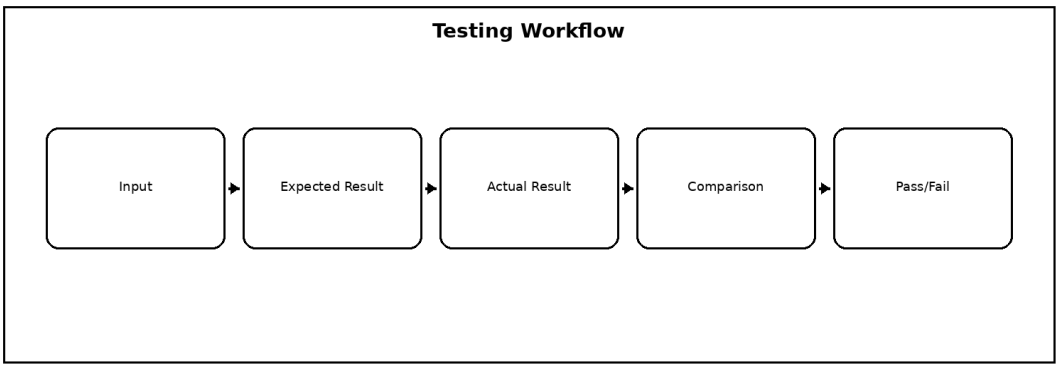


Fig. 5.15 Testing Workflow

5. RESULTS — User Workflow

The complete workflow from login through search, eligibility, recommendation, application, and tracking operates as an integrated placement process.

Search results can be improved by considering more than exact title equality. The system can inspect relevant job fields and use fuzzy similarity so that small typing mistakes do not prevent useful results from being returned. Ranking by similarity helps place the most relevant opportunities first.

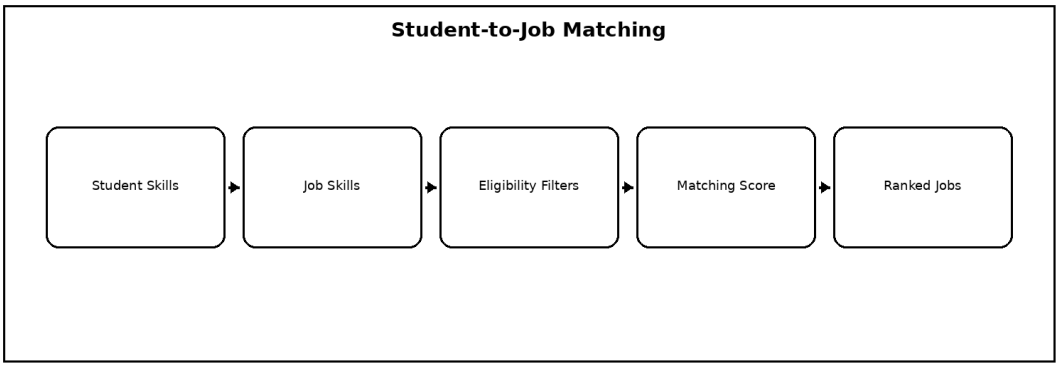


Fig. 5.16 Student-to-Job Matching Workflow

5. RESULTS — Summary

The results demonstrate that Placement Hub meets its main objective of applying DSA concepts to a realistic placement management problem while providing a usable browser interface.

The implemented workflow demonstrates that the project can manage student information, company records, job opportunities, search requests, recommendations, eligibility checks, and applications in one runtime session. The results section documents these outcomes through functional test cases and workflow descriptions.

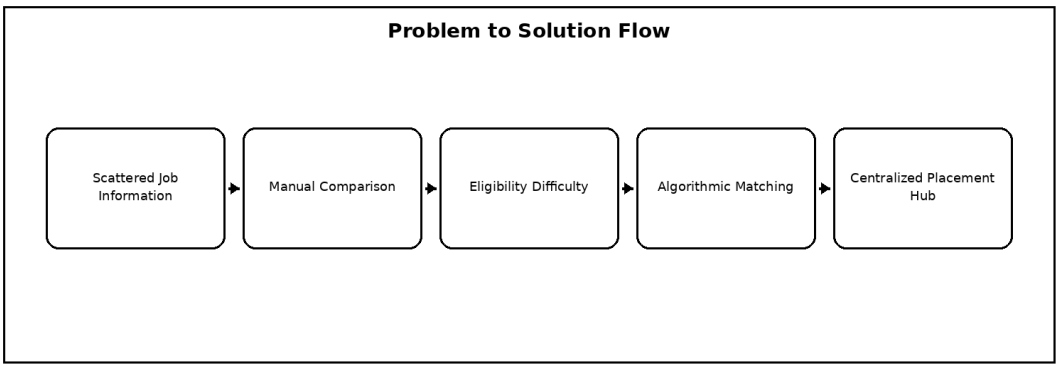


Fig. 5.17 Problem-to-Solution Flow

5. RESULTS — Academic Outcome

The implementation demonstrates practical use of collections, pattern matching, similarity measures, sorting, matching, and recommendation. The project therefore satisfies the intended DSA-3 learning outcomes.

The matching process can be viewed as a sequence of filters followed by ranking. Hard constraints such as branch, CGPA, and backlog limits remove unsuitable opportunities first. Skill similarity then contributes to the relevance of the remaining jobs, allowing the system to present more useful recommendations.

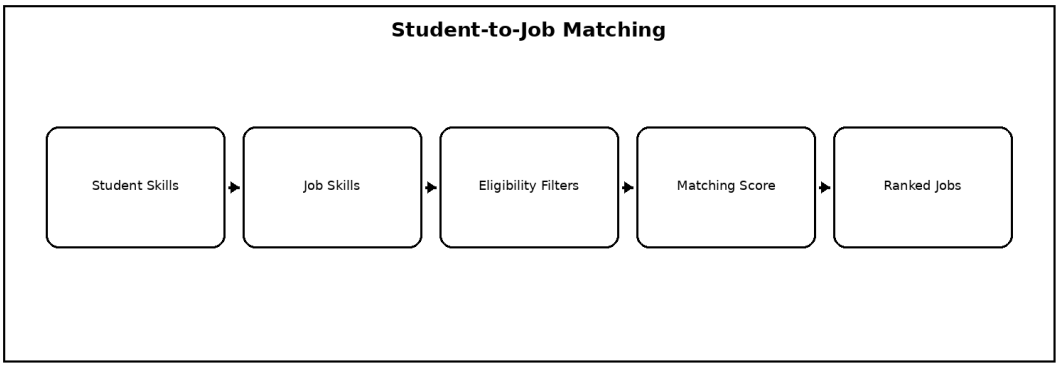


Fig. 5.18 Student-to-Job Matching Workflow

6. CHALLENGES & LIMITATIONS — 6.1 Issues Faced

A major challenge was integrating multiple DSA algorithms without making the project difficult to maintain. Separate DSA classes were used so that algorithms remain independent and reusable.

The DSA modules are not isolated textbook exercises; they are connected to placement operations. Pattern matching supports efficient text search, fuzzy methods tolerate spelling variation, sorting organizes results, and matching/recommendation logic combines multiple student and job attributes.

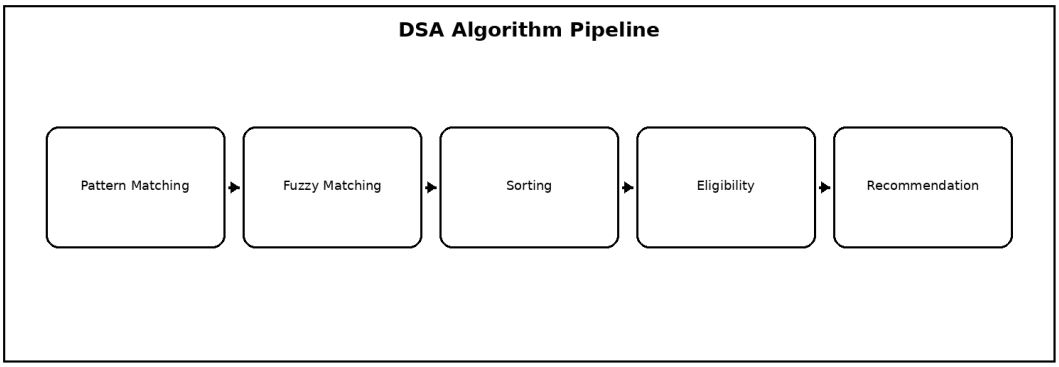


Fig. 6.1 DSA Algorithm Pipeline

6. CHALLENGES & LIMITATIONS — Data Consistency

Student, company, job, and application records are connected by IDs. Careful ID assignment and lookup methods were required to keep relationships consistent.

Most development challenges occurred at integration boundaries. A change in one model or DAO method could affect API responses or frontend rendering. These issues were addressed by keeping method responsibilities clear, compiling frequently, testing endpoints individually, and checking the complete browser workflow after major changes.

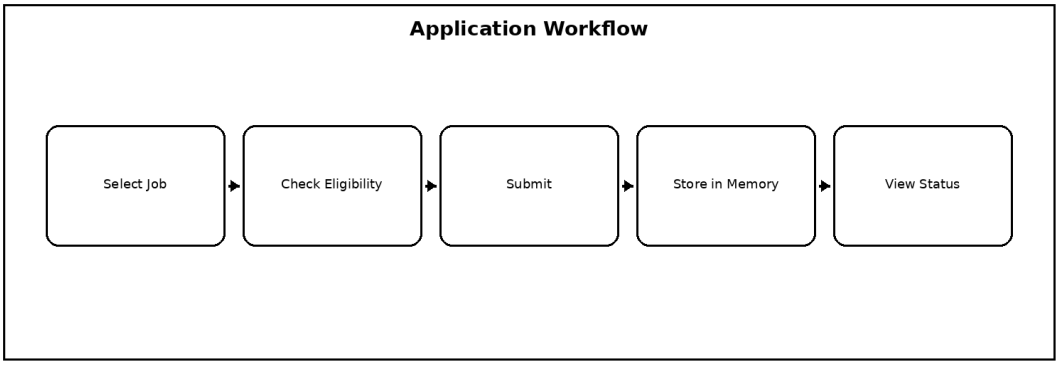


Fig. 6.2 Application Workflow

6. CHALLENGES & LIMITATIONS — Fuzzy Search

Fuzzy search required consistent handling of similarity thresholds. A percentage supplied by the interface must be converted correctly before numerical comparison.

Search results can be improved by considering more than exact title equality. The system can inspect relevant job fields and use fuzzy similarity so that small typing mistakes do not prevent useful results from being returned. Ranking by similarity helps place the most relevant opportunities first.

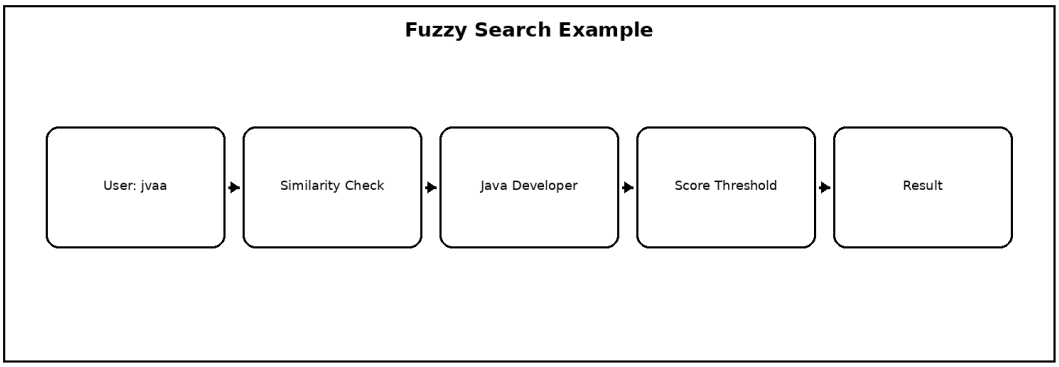


Fig. 6.3 Fuzzy Search Workflow

6. CHALLENGES & LIMITATIONS — Frontend Integration

Frontend JavaScript and backend API endpoints must agree on names, parameters, and response fields. Small changes in one layer can otherwise cause integration failures.

The frontend is designed around simple student actions. Forms collect input, dashboard sections expose major functions, and JavaScript calls the backend asynchronously. Results are rendered dynamically so that the user can search, inspect jobs, receive recommendations, and track applications without navigating through unrelated screens.

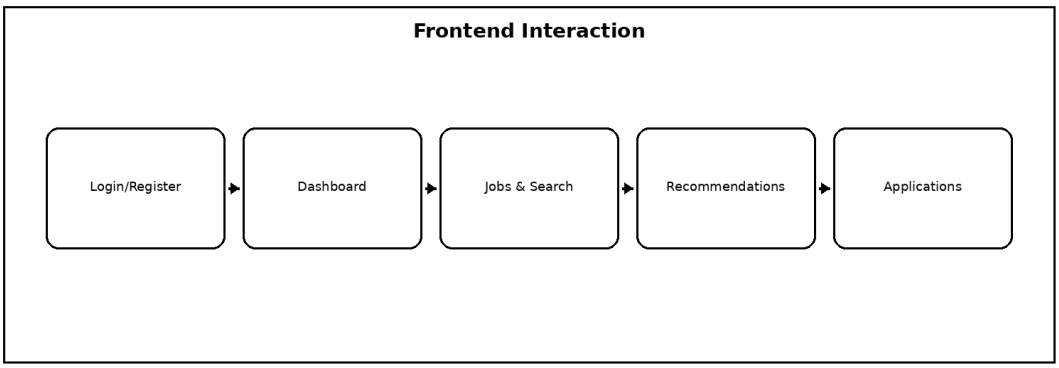


Fig. 6.4 Frontend Interaction

6. CHALLENGES & LIMITATIONS — Database Removal

The project originally used JDBC and MySQL. The academic requirement was changed to remove database usage, so DAO classes were redesigned to maintain runtime data using Java collections.

Most development challenges occurred at integration boundaries. A change in one model or DAO method could affect API responses or frontend rendering. These issues were addressed by keeping method responsibilities clear, compiling frequently, testing endpoints individually, and checking the complete browser workflow after major changes.

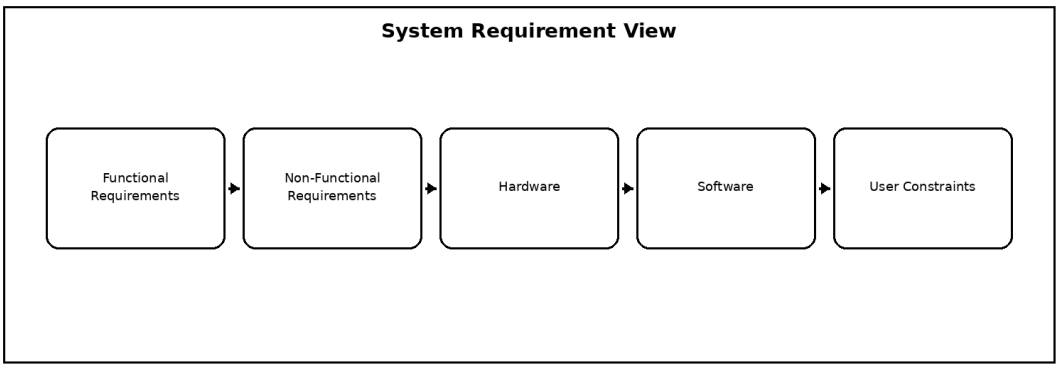


Fig. 6.5 System Requirements

6. CHALLENGES & LIMITATIONS — 6.2 Solutions Applied

The database dependency was removed while preserving DAO names and higher-level application workflows. This allowed the DSA project structure to remain stable while satisfying the database-free requirement.

Most development challenges occurred at integration boundaries. A change in one model or DAO method could affect API responses or frontend rendering. These issues were addressed by keeping method responsibilities clear, compiling frequently, testing endpoints individually, and checking the complete browser workflow after major changes.

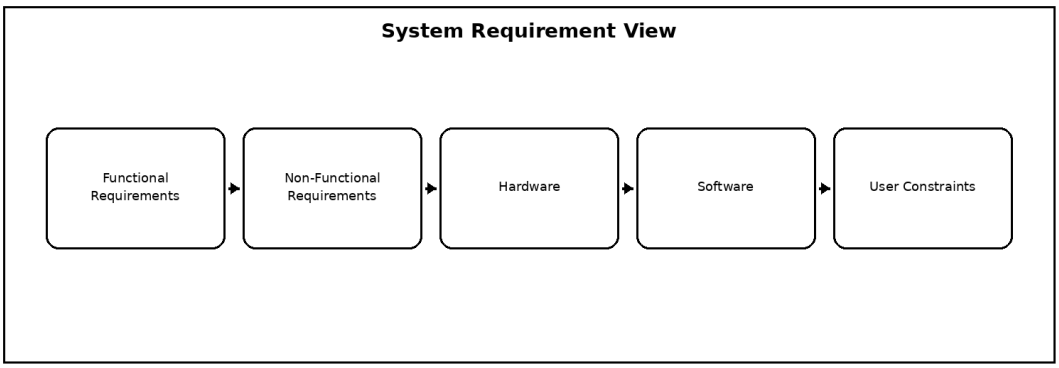


Fig. 6.6 System Requirements

6. CHALLENGES & LIMITATIONS — Algorithm Integration

Reusable search, fuzzy matching, sorting, and matching components were connected to the API layer. This made it possible to demonstrate algorithms through actual user actions.

The DSA modules are not isolated textbook exercises; they are connected to placement operations. Pattern matching supports efficient text search, fuzzy methods tolerate spelling variation, sorting organizes results, and matching/recommendation logic combines multiple student and job attributes.

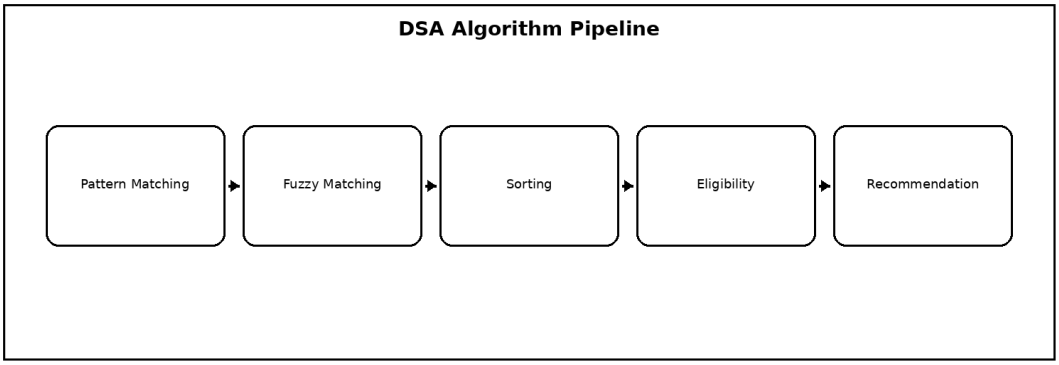


Fig. 6.7 DSA Algorithm Pipeline

6. CHALLENGES & LIMITATIONS — Testing Solution

Incremental compilation and testing were used after modifications to DAO, API, and DSA components. This reduced the risk of carrying multiple errors into the final integration.

The API layer provides a clear contract between the browser and Java backend. Endpoints accept operation-specific inputs and return structured results. This separation allows the frontend to remain focused on presentation while the backend handles validation, data access, and DSA processing.

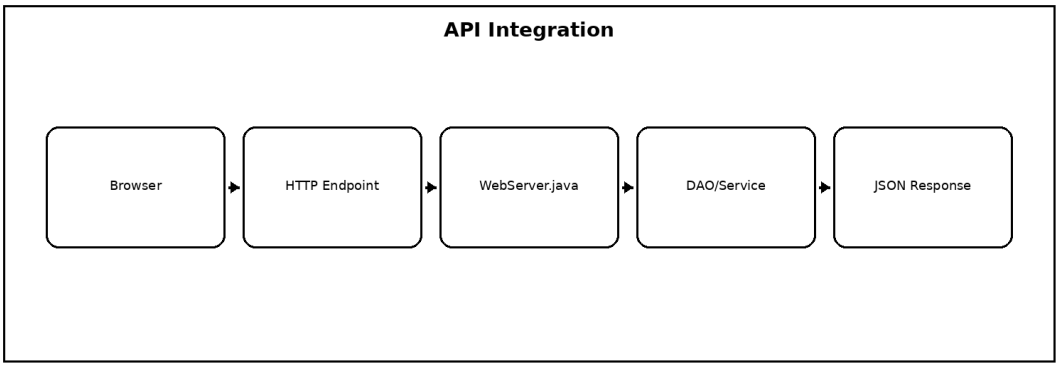


Fig. 6.8 API Integration

6. CHALLENGES & LIMITATIONS — 6.3 Limitations

Runtime data is not persistent. Newly registered students and submitted applications are lost when the server is restarted. This is an intentional limitation of the database-free version.

Most development challenges occurred at integration boundaries. A change in one model or DAO method could affect API responses or frontend rendering. These issues were addressed by keeping method responsibilities clear, compiling frequently, testing endpoints individually, and checking the complete browser workflow after major changes.

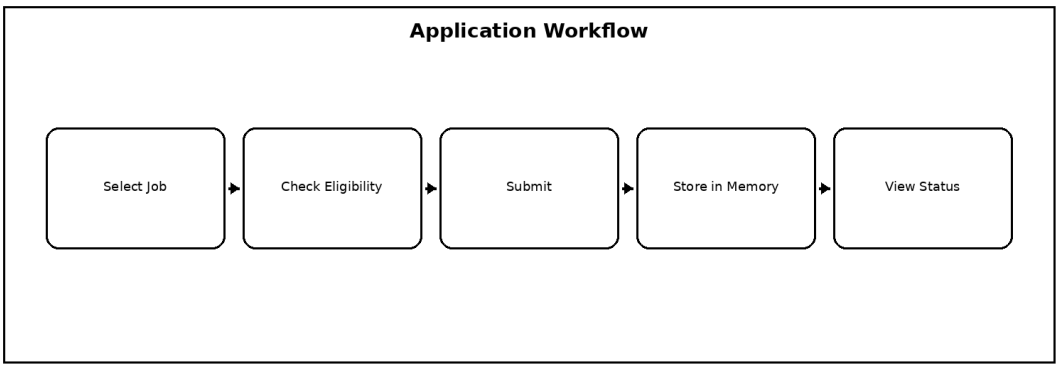


Fig. 6.9 Application Workflow

6. CHALLENGES & LIMITATIONS — Dataset Limitations

The 25 companies and 50 jobs are a demonstration dataset. They are not a live recruitment feed and should not be treated as current vacancy information.

Most development challenges occurred at integration boundaries. A change in one model or DAO method could affect API responses or frontend rendering. These issues were addressed by keeping method responsibilities clear, compiling frequently, testing endpoints individually, and checking the complete browser workflow after major changes.

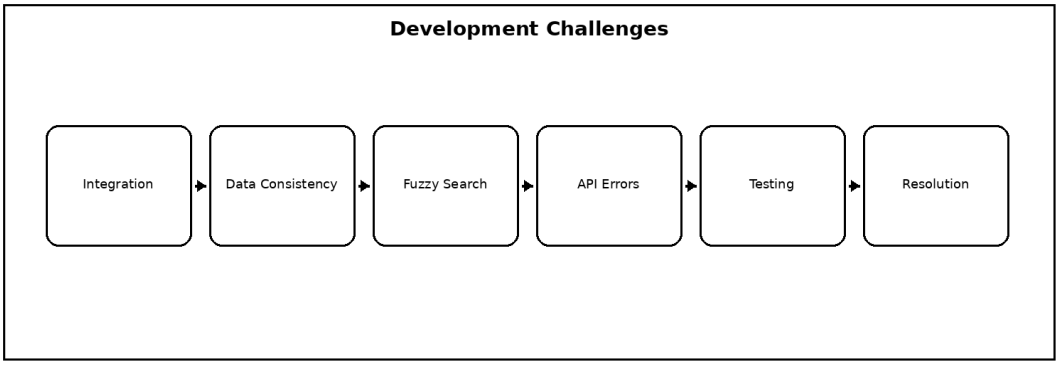


Fig. 6.10 Development Challenges

6. CHALLENGES & LIMITATIONS — Security Limitations

The academic prototype does not implement production-grade password hashing, token-based authentication, role-based authorization, or security monitoring.

Most development challenges occurred at integration boundaries. A change in one model or DAO method could affect API responses or frontend rendering. These issues were addressed by keeping method responsibilities clear, compiling frequently, testing endpoints individually, and checking the complete browser workflow after major changes.

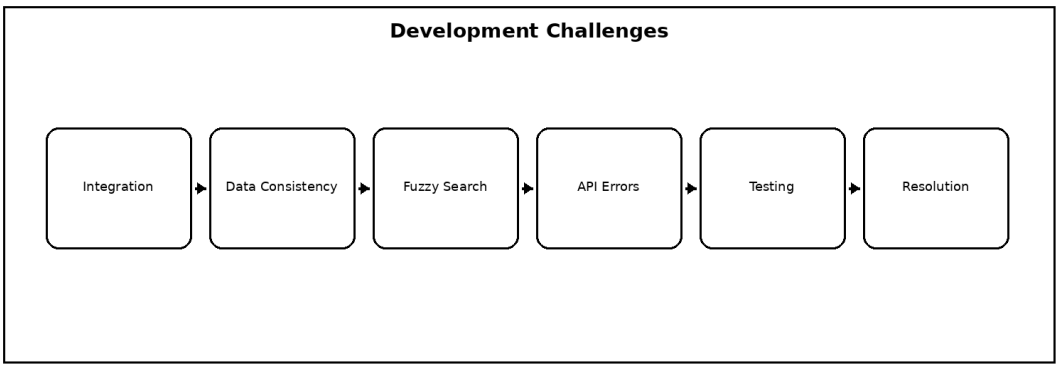


Fig. 6.11 Development Challenges

6. CHALLENGES & LIMITATIONS — Scalability

ArrayList-based storage is suitable for the project scale, but a production system with large datasets would require persistent storage, indexing, and optimized retrieval strategies.

Most development challenges occurred at integration boundaries. A change in one model or DAO method could affect API responses or frontend rendering. These issues were addressed by keeping method responsibilities clear, compiling frequently, testing endpoints individually, and checking the complete browser workflow after major changes.

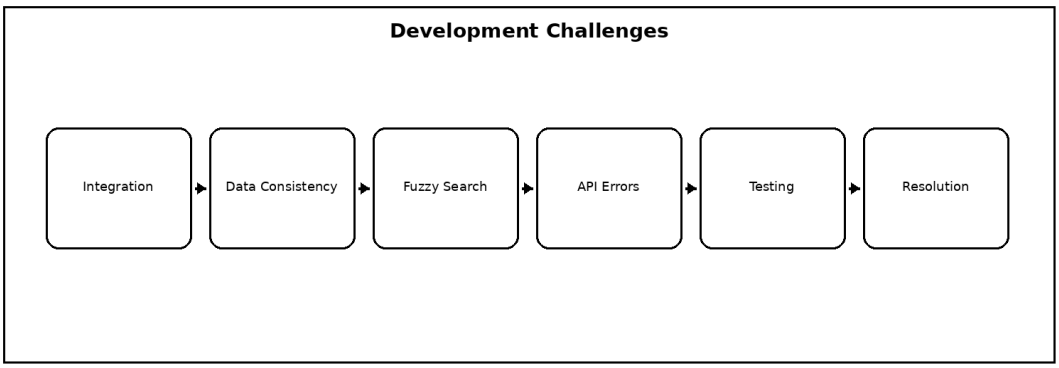


Fig. 6.12 Development Challenges

6. CHALLENGES & LIMITATIONS — Maintainability

The modular design improves maintainability, but automated unit tests, interfaces, configuration management, and logging would further strengthen the implementation.

Implementation is divided into small, testable units. Model classes represent data, DAO classes provide collection-based access, service classes coordinate higher-level operations, and DSA classes contain algorithmic logic. This modular structure also makes it easier to locate and correct errors during development.

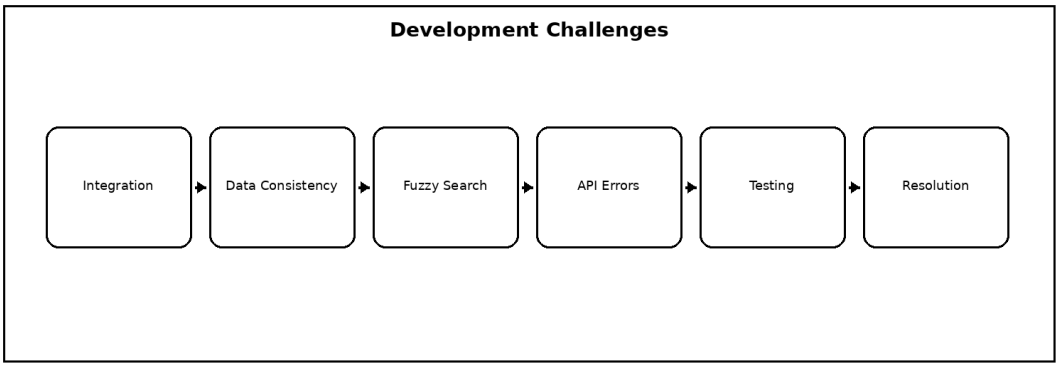


Fig. 6.13 Development Challenges

6. CHALLENGES & LIMITATIONS — Overall Assessment

The limitations do not prevent the project from meeting its academic objective. Instead, they identify clear opportunities for future engineering improvements.

Most development challenges occurred at integration boundaries. A change in one model or DAO method could affect API responses or frontend rendering. These issues were addressed by keeping method responsibilities clear, compiling frequently, testing endpoints individually, and checking the complete browser workflow after major changes.

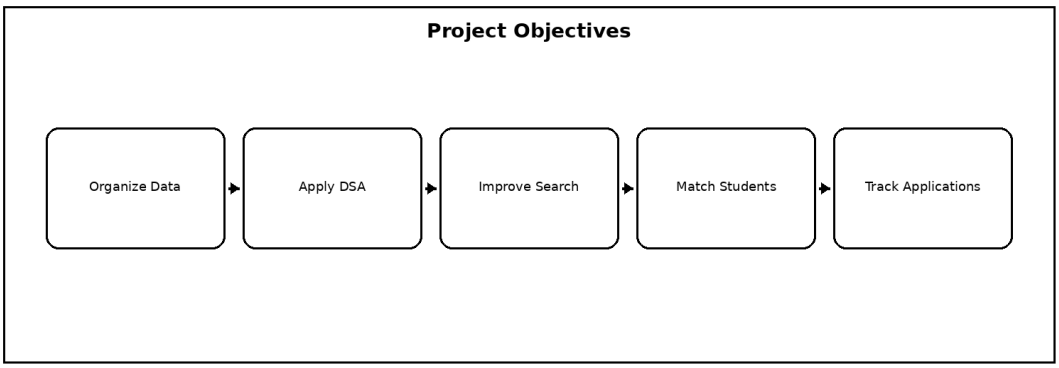


Fig. 6.14 Project Objectives

6. CHALLENGES & LIMITATIONS — Transition to Conclusion

The development process provided practical experience in converting a course requirement into a complete software workflow. The next chapter summarizes the final outcome and future directions.

The software stack was selected to keep the project easy to build and demonstrate. Java provides the object-oriented backend and DSA implementation, Maven handles compilation, and standard web technologies provide the user interface. The built-in Java HTTP server removes the need for an additional application server.

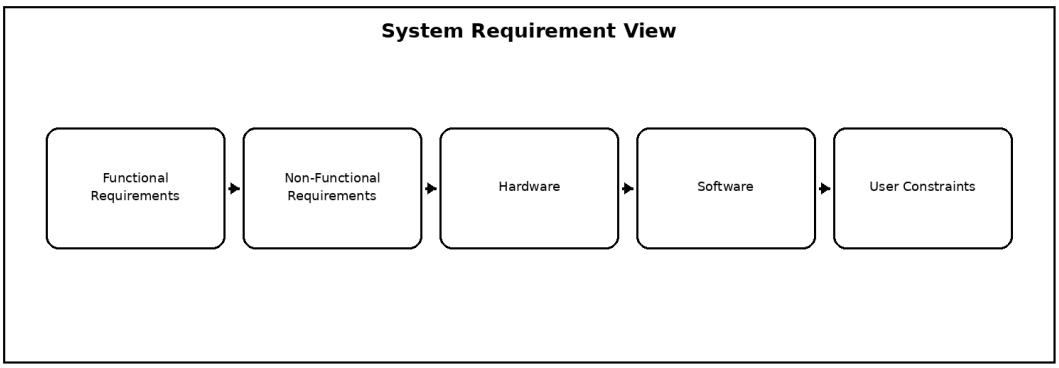


Fig. 6.15 System Requirements

7. CONCLUSION & FUTURE WORK — 7.1 Conclusion

Placement Hub successfully demonstrates a practical application of Data Structures and Algorithms in campus placement management. The system combines student management, company and job listings, search, fuzzy matching, eligibility, recommendation, and application tracking.

The choice of data structure depends on the operation. ArrayList is suitable for maintaining ordered collections of students, companies, jobs, and applications. HashMap supports key-value style access, while Stack and Queue demonstrate LIFO and FIFO processing. Trie and graph structures support more specialized algorithmic demonstrations.

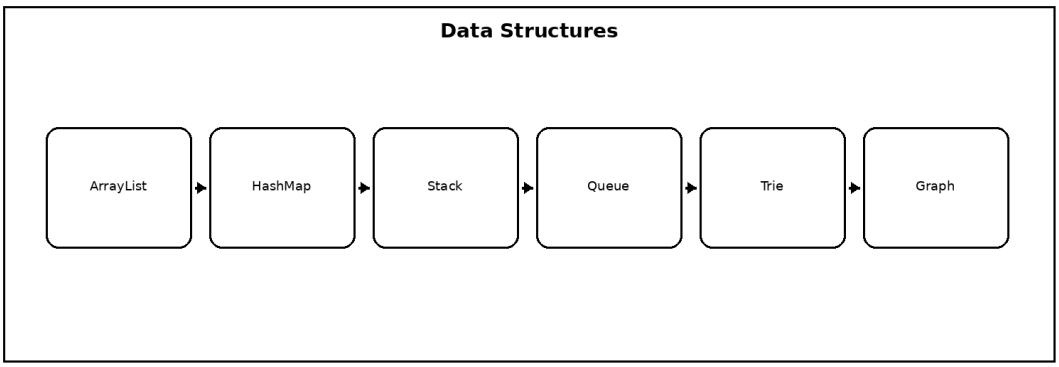


Fig. 7.1 Data Structures Used

7. CONCLUSION & FUTURE WORK — 7.1 Conclusion

The project shows how algorithms become meaningful when integrated into user-facing workflows. Pattern matching supports search, edit distance supports fuzzy matching, sorting organizes results, and matching logic supports recommendation and eligibility.

The DSA modules are not isolated textbook exercises; they are connected to placement operations. Pattern matching supports efficient text search, fuzzy methods tolerate spelling variation, sorting organizes results, and matching/recommendation logic combines multiple student and job attributes.

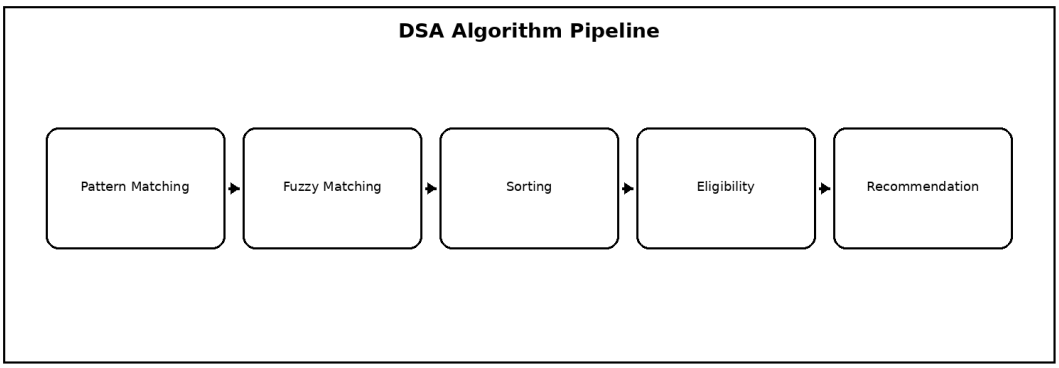


Fig. 7.2 DSA Algorithm Pipeline

7. CONCLUSION & FUTURE WORK — Architecture Summary

The layered architecture separates frontend, API, DAO, service, model, and DSA responsibilities. This makes the system easier to explain, test, and extend.

The layered design reduces coupling between the user interface and algorithmic implementation. The WebServer layer is responsible for HTTP routing, DAO classes manage runtime records, service classes coordinate application logic, and DSA classes perform algorithmic processing. This organization supports both demonstration and maintenance.

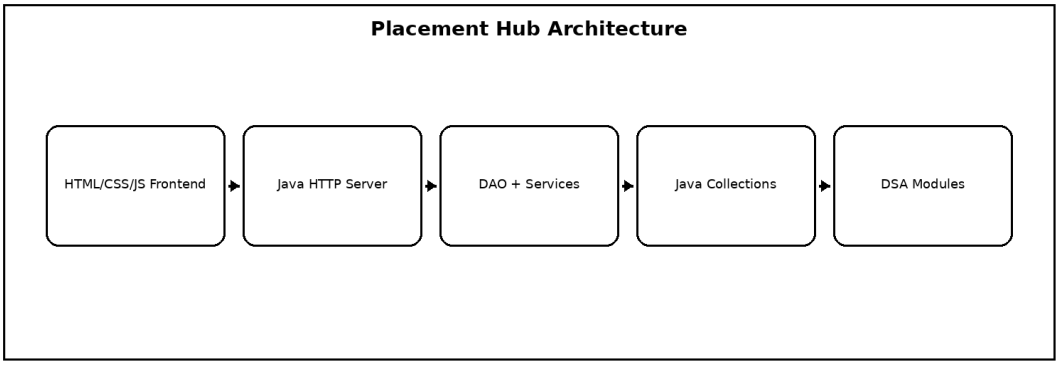


Fig. 7.3 High-Level System Architecture

7. CONCLUSION & FUTURE WORK — 7.2 Future Work

Persistent database or cloud storage can be added in a future version. The DAO layer provides a natural abstraction point for introducing persistence without redesigning the complete application.

Future improvements can preserve the existing architecture while replacing individual components. Persistent storage could be added behind the DAO layer, while the recommendation component could later use machine-learning models. Additional authentication, analytics, live placement feeds, and cloud deployment could be added without changing the core student-facing workflow.

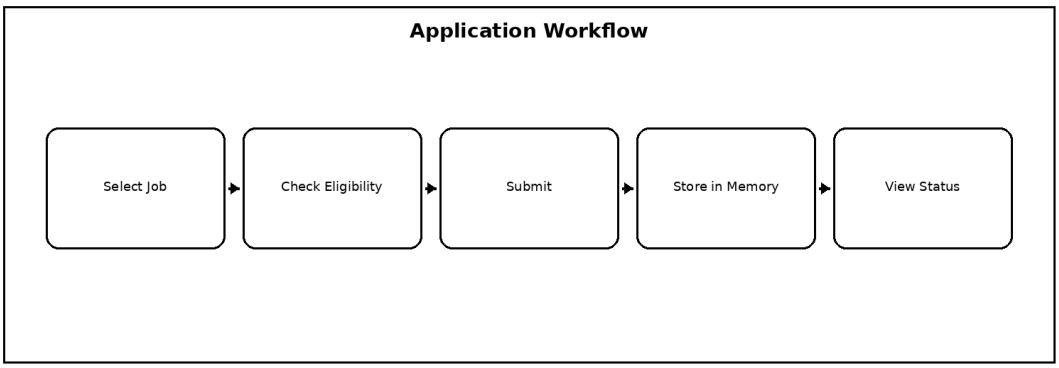


Fig. 7.4 Application Workflow

7. CONCLUSION & FUTURE WORK — Future Recommendation

Machine-learning or learning-to-rank techniques could improve recommendation quality by learning from student interests, applications, and placement outcomes.

Recommendations are generated from the relationship between a student's profile and the available jobs. Academic constraints are considered before relevance is ranked. This approach avoids recommending opportunities that are clearly outside the student's eligibility and focuses the final list on actionable choices.

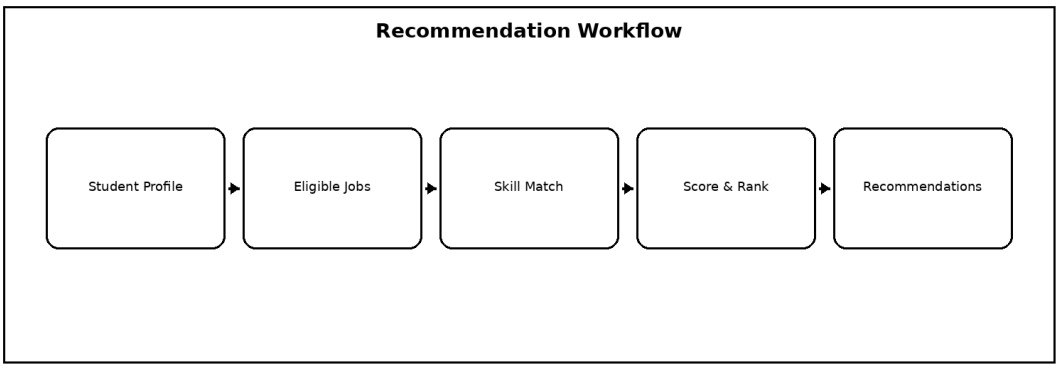


Fig. 7.5 Recommendation Workflow

7. CONCLUSION & FUTURE WORK — Future Platform Features

Future versions can include placement-officer dashboards, company-side job posting, notifications, recruitment calendars, analytics, role-based access, and cloud deployment.

Future improvements can preserve the existing architecture while replacing individual components. Persistent storage could be added behind the DAO layer, while the recommendation component could later use machine-learning models. Additional authentication, analytics, live placement feeds, and cloud deployment could be added without changing the core student-facing workflow.

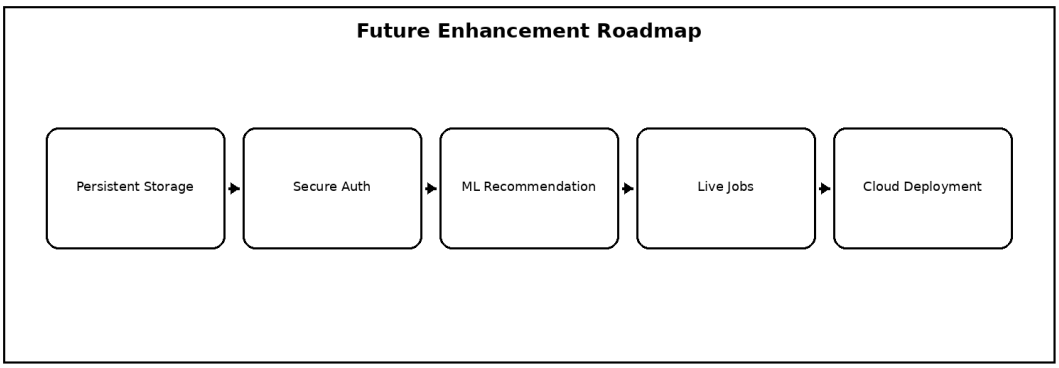


Fig. 7.6 Future Enhancement Roadmap

7. CONCLUSION & FUTURE WORK — 7.3 Skills Learned

The team gained experience in Java, object-oriented design, collections, searching, sorting, pattern matching, string similarity, HTTP APIs, frontend development, testing, debugging, documentation, and teamwork.

The matching process can be viewed as a sequence of filters followed by ranking. Hard constraints such as branch, CGPA, and backlog limits remove unsuitable opportunities first. Skill similarity then contributes to the relevance of the remaining jobs, allowing the system to present more useful recommendations.

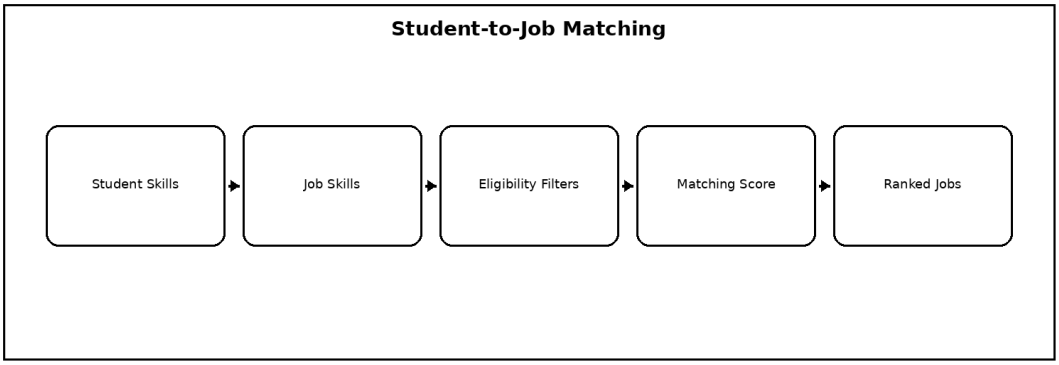


Fig. 7.7 Student-to-Job Matching Workflow

7. CONCLUSION & FUTURE WORK — Learning Reflection

The project strengthened understanding of how algorithmic choices affect application behavior. It also reinforced the importance of modular design, validation, API contracts, and incremental testing.

The API layer provides a clear contract between the browser and Java backend. Endpoints accept operation-specific inputs and return structured results. This separation allows the frontend to remain focused on presentation while the backend handles validation, data access, and DSA processing.

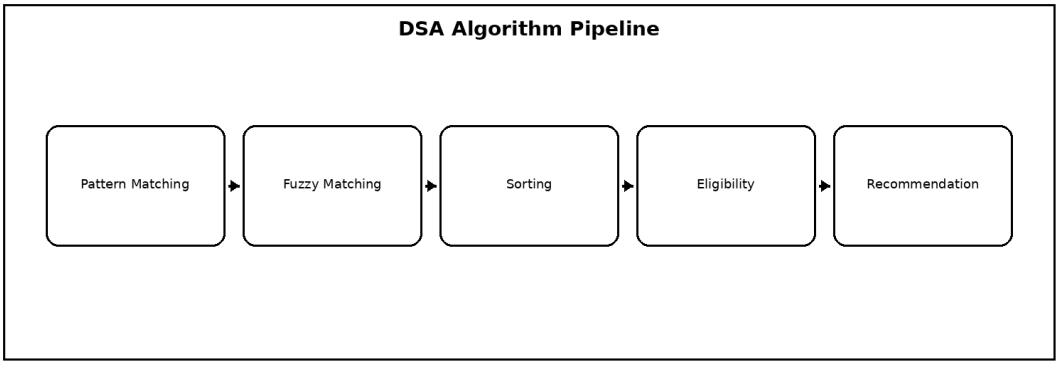


Fig. 7.8 DSA Algorithm Pipeline

7. CONCLUSION & FUTURE WORK — Final Summary

Placement Hub meets the academic objective of demonstrating DSA concepts through a realistic placement management application. It provides a foundation that can be extended into a persistent and production-oriented placement platform.

Future improvements can preserve the existing architecture while replacing individual components. Persistent storage could be added behind the DAO layer, while the recommendation component could later use machine-learning models. Additional authentication, analytics, live placement feeds, and cloud deployment could be added without changing the core student-facing workflow.

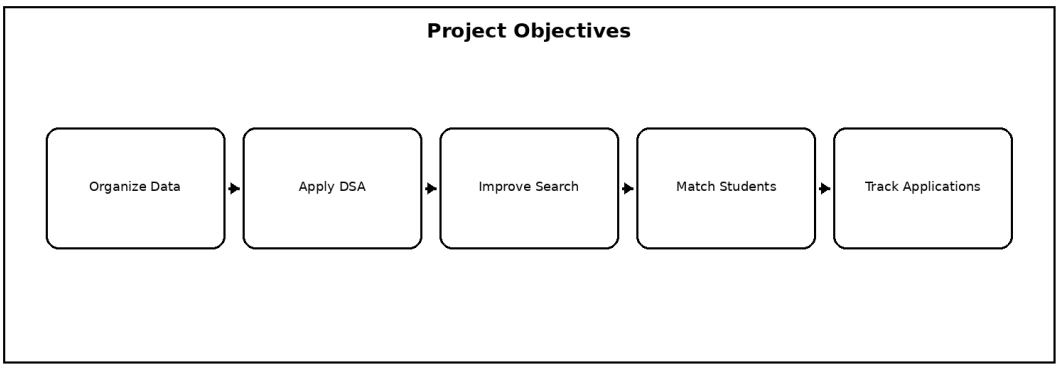


Fig. 7.9 Project Objectives

REFERENCES

[1] Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest and Clifford Stein, Introduction to Algorithms, MIT Press.

[2] Robert Lafore, Data Structures and Algorithms in Java, Pearson.

[3] Herbert Schildt, Java: The Complete Reference, McGraw-Hill.

[4] Oracle, Java Platform, Standard Edition Documentation.

[5] Oracle, Java Collections Framework Documentation.

REFERENCES — CONTINUED

[6] Apache Maven Project, Maven Documentation.

[7] MDN Web Docs, HTML, CSS and JavaScript Documentation.

[8] Java API documentation for com.sun.net.httpserver and standard Java networking classes.

[9] Data Structures and Algorithms-3 course materials and laboratory guidance.

[10] Placement Hub project source code, test cases, and implementation notes prepared by the project team.